



UNIVERSIDAD DE MÁLAGA



Graduado en Ingeniería Informática

Quantum Proxy: Plataforma para elegir y gestionar
proveedores de computación cuántica
heterogéneos

Quantum Proxy: Platform for choosing and managing
heterogeneous quantum computing providers

Realizado por
Fabrizio Cristallo Cagnoli

Tutorizado por
José Carlos Canal Velasco
José Manuel García Alonso

Departamento
Lenguajes y Ciencias de la Comunicación

MÁLAGA, septiembre de 2024



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADUADA/O EN TECNOLOGÍAS DE LA INFORMACIÓN

**Quantum Proxy: Plataforma para elegir y gestionar
proveedores de computación cuántica heterogéneos**

**Quantum Proxy: Platform for choosing and managing
heterogeneous quantum computing providers**

Realizado por
Fabrizio Cristallo Cagnoli

Tutorizado por
José Carlos Canal Velasco
José Manuel García Alonso

Departamento
Lenguajes y Ciencias de la Comunicación

UNIVERSIDAD DE MÁLAGA
MÁLAGA, SEPTIEMBRE DE 2024

Fecha defensa: septiembre de 2024

Resumen

El propósito de este trabajo se centra en el desarrollo de *Quantum Proxy*, una plataforma web diseñada para abordar la fragmentación existente entre los distintos proveedores de computación cuántica. El objetivo principal del proyecto es facilitar la interacción de los usuarios con los diferentes proveedores mediante una interfaz unificada que simplifique la selección de sistemas cuánticos en función de las necesidades específicas del usuario. Para ello, la plataforma presenta un asistente de elección y un comparador de máquinas cuánticas, además de un sistema de automatización para la actualización de datos de los proveedores. El sistema se ha basado en una arquitectura modular, con la integración de tecnologías como FastAPI, Angular, y MongoDB, que permiten una fácil extensión de la plataforma. El desarrollo de esta plataforma supone una mejora en la experiencia de usuario al centralizar la información de varios proveedores y asistir en la toma de decisiones, reduciendo la complejidad técnica. Aunque el problema de la diversidad tecnológica sigue presente, la plataforma permite mitigar en parte la barrera de entrada para usuarios menos experimentados. En conclusión, el proyecto cumple su propósito de simplificar la interacción con el ecosistema cuántico y ofrece una base sólida para futuras mejoras y expansiones.

Palabras clave: Computación cuántica, Proveedores cuánticos, Desarrollo web, Automatización

Abstract

The purpose of this work focuses on the development of *Quantum Proxy*, a web platform designed to address the fragmentation among different quantum computing providers. The main objective of the project is to simplify user interaction with various providers through a unified interface that streamlines the selection of quantum systems based on the user's specific needs. To achieve this, the platform features a selection assistant and a quantum machine comparison tool, as well as an automated system for updating provider data. The system is built on a modular architecture, integrating technologies such as FastAPI, Angular, and MongoDB, allowing for easy platform extension. The development of this platform improves the user experience by centralizing information from various providers and assisting in decision-making, thereby reducing technical complexity. Although the issue of technological diversity remains, the platform helps mitigate entry barriers for less experienced users. In conclusion, the project successfully fulfills its goal of simplifying interaction with the quantum ecosystem and provides a solid foundation for future improvements and expansions.

Keywords: Quantum computing, Quantum providers, Web development, Automation

Índice

1. Introducción	7
1.1. Motivación	7
1.2. Objetivos	8
1.3. Resultados	9
1.4. Estructura del documento	10
2. Tecnologías Empleadas	11
2.1. Backend	11
2.1.1. Python + FastAPI	11
2.1.2. Selenium	11
2.1.3. MongoDB + MongoDB Compass	12
2.2. Frontend	12
2.2.1. Angular	12
2.2.2. Bootstrap	13
2.3. Otras Tecnologías	13
2.3.1. Docker	13
2.3.2. Render, Vercel y MongoDB Atlas	13
2.3.3. Visual Studio Code	14
2.3.4. Trello	14
2.3.5. GitHub	14
3. Estado del Arte	15
3.1. Computación Cuántica	15
3.2. Situación y Problemática	17
3.3. Proveedores Cuánticos	17
3.3.1. Amazon Braket	18
3.3.2. IBM Quantum	19
3.3.3. IonQ	21

3.3.4. Rigetti	22
3.4. Soluciones y Alternativas Actuales	22
3.5. Qué aporta <i>Quantum Proxy</i> en este contexto	23
4. Metodología de Trabajo	25
4.1. Iteración 1 (Semanas 1-3)	26
4.2. Iteración 2 (Semanas 4-6)	27
4.3. Iteración 3 (Semanas 7-8)	27
4.4. Iteración 4 (Semanas 9-11)	28
4.5. Iteración 5 (Semanas 12-13)	28
4.6. Iteración 6 (Semanas 14-15)	29
4.7. Iteración 7 (Semanas 16-18)	30
4.8. Iteración 8 (Semanas 19-21)	30
4.9. Iteración Final (Semanas 22-27)	30
5. Arquitectura y Modelado del Sistema	33
5.1. Visión General de la Arquitectura	33
5.2. Arquitectura del Backend	35
5.3. Arquitectura del Frontend	35
5.4. Modelo de Datos y Estructura de la Base de Datos	37
5.4.1. Modelos para la gestión de usuarios	37
5.4.2. Modelos para la gestión de proveedores	38
5.4.3. Modelos para la gestión de sistemas	40
6. Diseño y Organización en Módulos	45
6.1. Visión general del Backend	45
6.2. Módulo para la recopilación de datos	46
6.2.1. Integración con la base de datos	46
6.2.2. Aplicación directa en el código	48
6.3. Recopilación: módulos del sistema	50
6.3.1. Módulos del Frontend	51
6.3.2. Módulos del Backend	51

6.3.3. Módulos de la Base de Datos	52
7. Extensibilidad	53
7.1. Requisitos para Extender el Sistema	53
7.2. Pasos para Extender el Sistema	53
7.2.1. Configuración en la Base de Datos	54
7.2.2. Reconfiguración del sistema	55
7.3. Eliminar un Proveedor del Sistema	56
8. Despliegue de la Aplicación	57
8.1. Despliegue en la Nube	57
8.1.1. Backend con Render	58
8.1.2. Frontend con Vercel	59
8.1.3. Base de Datos con MongoDB Atlas	59
8.2. Despliegue en Local	59
9. Conclusiones y Líneas Futuras	63
9.1. Conclusiones	63
9.1.1. Reflexión personal	63
9.1.2. Conclusiones del proyecto	63
9.2. Líneas Futuras	64
Bibliografía	67
Apéndice A. Manual de Uso	69
A.1. Página principal	69
A.2. Registro	70
A.3. Inicio de sesión	71
A.4. Recuperación de contraseña	72
A.5. Menú desplegable para usuarios autenticados	73
A.5.1. My Jobs	74
A.5.2. Account Settings	76
A.5.3. Admin Tools	77

A.6. Selección de tema de color	80
A.7. Barra de navegación	81
A.7.1. Providers	81
A.7.2. Systems	82
A.7.3. Compare	84
A.7.4. Wizard	85
A.8. Acceso privilegiado a la plataforma	86
Apéndice B. Manual de Instalación	87
B.1. Requisitos previos a la instalación	87
B.2. Descarga del código fuente	87
B.3. Configuración del archivo docker-compose.yml	87
B.3.1. Servicio <i>database</i>	89
B.3.2. Servicio <i>backend</i>	90
B.3.3. Servicio <i>frontend</i>	92
B.4. Manejar los servicios con Docker Compose	94
B.4.1. Construcción de los servicios	94
B.4.2. Acceso a la plataforma recién desplegada	94
B.4.3. Terminación de los servicios	95
B.4.4. Siguiendo pasos a tomar	95

1

Introducción

1.1. Motivación

La computación cuántica es una tecnología emergente que promete revolucionar numerosos campos, desde la ciberseguridad hasta la optimización de procesos industriales. Sin embargo, uno de los principales desafíos para su adopción masiva es la fragmentación tecnológica entre los distintos proveedores de servicios cuánticos. Cada proveedor ofrece infraestructuras, lenguajes de programación y herramientas de desarrollo diferentes, lo que genera un entorno heterogéneo y complejo para los desarrolladores y empresas que desean aprovechar el poder de la computación cuántica.

Este entorno fragmentado dificulta la toma de decisiones informadas por parte de los usuarios que buscan el proveedor cuántico más adecuado para sus necesidades específicas. La gran diversidad de opciones y la falta de estandarización hacen que el proceso de seleccionar y aprender a utilizar un proveedor sea costoso y consume tiempo, limitando la capacidad de los usuarios para explorar el potencial de la computación cuántica.

Este TFG se motiva en la necesidad de crear una solución que ayude a los usuarios a moverse dentro de esta diversidad, ofreciendo una guía o asistente que facilite la selección del proveedor cuántico más adecuado según sus necesidades. Aunque no elimina la heterogeneidad ni la incompatibilidad entre los diferentes proveedores, la solución propuesta pretende simplificar el proceso de elección, al centralizar en un único punto la información de diversos proveedores. Además, a pesar de que el problema del “vendor lock-in” (o dependencia de un único proveedor) sigue presente, la propuesta permite mitigar parcialmente este problema al ofrecer una herramienta que asista en la elección informada del proveedor, mejorando así la experiencia del usuario y fomentando una adopción más amplia de la computación cuántica.

1.2. Objetivos

El objetivo principal de este trabajo es desarrollar una plataforma web que actúe como un punto en común para diversos proveedores, integrando de manera homogénea la información ofrecida por cada uno. Y puesto que otra problemática es la gran variabilidad del mercado (proveedores nuevos que aparecen, máquinas que son retiradas, etc.), la plataforma planteará un modelo flexible que permita añadir o retirar proveedores en función del mercado tecnológico.

Para ello, se propone un sistema automatizado que mediante el uso de consultas API, kits de desarrollo (SDKs), o *web scraping*, permita extraer los datos y máquinas ofrecidas por cada uno de los proveedores designados.

Con todo esto, la plataforma permitirá a los desarrolladores y usuarios abstraer las diferencias tecnológicas entre los proveedores, facilitando la elección de un ordenador cuántico y el acceso a los datos ofrecidos por estos.

Para lograr este objetivo principal, se plantean los siguientes objetivos específicos:

- **Centralización de los Datos:** Desarrollar una API que centralice el acceso a la información y máquinas ofrecidas por distintos proveedores de computación cuántica, permitiendo gestionar todos los servicios desde un único punto.
- **Automatización del Sistema:** Incorporar módulos que automaticen la recogida de datos de cada proveedor, usando la herramienta más adecuada en cada caso.
- **Extensibilidad del Sistema:** Ofrecer flexibilidad a la hora de incorporar nuevos proveedores, minimizando la cantidad de código a modificar.
- **Facilidad de Uso:** Diseñar una interfaz de usuario intuitiva que permita a los desarrolladores interactuar con los servicios cuánticos de manera sencilla, sin necesidad de conocimientos profundos sobre cada plataforma específica.
- **Asistencia al Usuario:** Mejorar y asistir la búsqueda de una máquina que cumpla con las necesidades del usuario, basándose en criterios como el número de *qubits*, tiempo de cola y precios.
- **Envío de Trabajos:** Se permitirá en la medida de lo posible el envío de circuitos cuánticos al proveedor deseado, pudiendo consultar el resultado de estos.

Cabe destacar que se ha querido aprovechar este trabajo para indagar en conocimientos relativos al desarrollo web, experimentando con nuevas tecnologías, cuidando al detalle cada funcionalidad, y buscando proporcionar una buena experiencia de usuario. Se profundizará más sobre este tema en el apartado 9.1 de Conclusiones.

1.3. Resultados

Siguiendo tales objetivos, los resultados obtenidos han sido los siguientes:

- Plataforma Web — La cual unifica y centraliza datos y máquinas de distintos proveedores, abstrayendo la inherente complejidad entre ellos. En ella se incluyen los siguientes servicios:
 - Acceso centralizado a proveedores.
 - Acceso centralizado a máquinas cuánticas.
 - Comparativa entre proveedores y máquinas cuánticas.
 - *Wizard* (o Asistente) para ayudar al usuario en su búsqueda por una máquina que satisfaga sus necesidades.
- Despliegue en la Nube — La plataforma ha sido desplegada usando tecnologías como Docker, Render y Vercel para asegurar que la aplicación sea accesible por todo el mundo.
- Código Fuente — El código utilizado en el desarrollo de la plataforma, alojado en un repositorio de GitHub.
- Documentación Técnica — Se incluye un manual de usuario y una guía de instalación, detallando cómo interactuar con la plataforma y cómo desplegarla en un entorno local para desarrollo.
- Memoria — Documenta todo el proceso seguido para la realización del TFG, desde la investigación inicial, pasando por el desarrollo de la aplicación, hasta su despliegue en la nube.

1.4. Estructura del documento

- El capítulo 1 describe la motivación del trabajo, sus objetivos, y los resultados obtenidos.
- El capítulo 2 presenta las tecnologías y herramientas utilizadas en el proyecto.
- El capítulo 3 introduce el contexto de la computación cuántica, realiza un estudio sobre la situación actual y principales herramientas existentes hoy día, y analiza los proveedores de los cuales se extraerán los datos.
- El capítulo 4 presenta la metodología de trabajo seguida en este TFG, detallando los avances de cada iteración.
- El capítulo 5 detalla la arquitectura del sistema, separando *backend* y *frontend*, junto con los modelos utilizados para la base de datos.
- El capítulo 6 describe el funcionamiento de los distintos módulos, encargados de contribuir directa o indirectamente a la recolecta de datos de manera automática.
- El capítulo 7 expone los pasos a seguir dado el caso que se quiera añadir nuevos proveedores al sistema.
- El capítulo 8 explica el proceso por el cual la aplicación es empaquetada usando *Docker*, para proporcionar un despliegue íntegro tanto en la nube como en un entorno local.
- El capítulo 9 concluye con las conclusiones sacadas del proyecto, así como líneas futuras y posibles expansiones del proyecto.
- El apéndice A presenta el manual de uso de la plataforma.
- El apéndice B presenta el manual de instalación para continuar el desarrollo de la aplicación en un entorno local.

2

Tecnologías Empleadas

2.1. Backend

El backend constituye el núcleo de la aplicación, donde se describe la lógica de negocio que permitirá a los distintos módulos interactuar entre sí. Estas son algunas de las tecnologías y herramientas destacables para su desarrollo:

2.1.1. Python + FastAPI

Python [1] es un lenguaje de programación interpretado, de alto nivel y propósito general, conocido por su simplicidad y legibilidad, utilizado ampliamente en el desarrollo web, análisis de datos e inteligencia artificial. Python constituirá la base de la lógica de negocio y backend, aprovechando su vasta colección de bibliotecas y facilidad de integración para manejar la lógica compleja y agilizar el desarrollo de la plataforma.

Por otro lado, FastAPI [2] es un framework moderno y rápido para construir APIs con Python, basado en los estándares de OpenAPI y JSON Schema. Facilita la creación eficiente de APIs RESTful, con validación automática de datos y documentación interactiva integrada. FastAPI permitirá desarrollar una API escalable y de alto rendimiento para gestionar tanto usuarios como proveedores dentro de la aplicación, ofreciendo a su vez una gran experiencia de desarrollo.

2.1.2. Selenium

Selenium [3] es una herramienta para la automatización de navegadores web, que permite interactuar con páginas web como lo haría un usuario real. Se utiliza principalmente para

automatizar pruebas y realizar *web scraping*. Selenium se empleará para extraer información de proveedores de computación cuántica cuando no se disponga de APIs o SDKs oficiales, asegurando así que la plataforma tenga datos actualizados y relevantes.

2.1.3. MongoDB + MongoDB Compass

MongoDB [4] es una base de datos NoSQL orientada a documentos que almacena datos en un formato flexible, similar a JSON, lo que le permite manejar información no estructurada o semiestructurada sin necesidad de un esquema fijo. La elección de un modelo no relacional es fundamental para un proyecto como este, dada la heterogeneidad entre proveedores y la diversidad de los datos.

Por otro lado, MongoDB Compass [5] es una interfaz gráfica de usuario (GUI) para MongoDB, que facilita la gestión y visualización de datos, permitiendo consultas y análisis de manera interactiva. MongoDB Compass servirá para administrar y gestionar la base de datos de una manera mucho más sencilla e intuitiva.

2.2. Frontend

El frontend es la parte de la aplicación que interactúa directamente con el usuario final. Su objetivo es proporcionar una interfaz intuitiva y eficiente que permita a los usuarios acceder a las funcionalidades del sistema de manera sencilla y rápida.

2.2.1. Angular

Angular [6] es un framework de desarrollo de aplicaciones web basado en TypeScript, mantenido por Google, que permite crear aplicaciones dinámicas y robustas. Con características modernas como la gestión del estado y el enrutamiento, Angular se utilizará para construir el frontend de la aplicación, proporcionando una interfaz de usuario interactiva y receptiva para gestionar y visualizar los datos de proveedores y servicios de computación cuántica de manera intuitiva.

2.2.2. Bootstrap

Bootstrap [7] es un framework de diseño web que facilita la creación de sitios web y aplicaciones web responsivas y visualmente atractivas por medio de componentes predefinidos de HTML, CSS y JavaScript que son fácilmente personalizables. Bootstrap se usará para facilitar el desarrollo del frontend, haciendo uso de los distintos componentes y clases que este ofrece para personalizar al gusto.

2.3. Otras Tecnologías

2.3.1. Docker

Docker [8] es una plataforma para desarrollar, enviar y ejecutar aplicaciones dentro de contenedores, que son entornos virtuales ligeros y portables. Facilita la creación de entornos de desarrollo consistentes y el despliegue de aplicaciones en múltiples entornos. Docker se utilizará para empaquetar la aplicación y sus dependencias en contenedores, lo que facilitará el despliegue en entornos de producción, garantizando portabilidad y consistencia.

2.3.2. Render, Vercel y MongoDB Atlas

Render [9] es una plataforma de hosting en la nube que facilita el despliegue y escalado de aplicaciones web, APIs y contenedores Docker, ofreciendo un entorno automatizado con despliegue continuo. Render se usará para el despliegue del backend, asegurando que la API y otros servicios estén disponibles en la nube con una gestión simplificada del entorno de producción.

Vercel [10] es una plataforma en la nube especializada en el despliegue de aplicaciones frontend y sitios web estáticos, optimizada para frameworks como Angular y React. Con características como CDN integrado y despliegue continuo, Vercel se utilizará para el despliegue del frontend de la aplicación, tomando ventaja del despliegue continuo antes mencionado.

MongoDB Atlas [11] es una plataforma de base de datos como servicio (DBaaS) que ofrece un entorno completamente gestionado para MongoDB en la nube. MongoDB Atlas se utilizará para desplegar la base de datos de manera sencilla y eficiente.

2.3.3. Visual Studio Code

Visual Studio Code [12] (VS Code) es un editor de código fuente ligero pero potente, desarrollado por Microsoft, que soporta una amplia gama de lenguajes de programación y herramientas de desarrollo. Con características como depuración integrada, control de versiones y un terminal incorporado, VS Code será el entorno de desarrollo principal, proporcionando herramientas avanzadas para escribir, depurar y administrar el código.

2.3.4. Trello

Trello [13] es una herramienta de gestión de proyectos que utiliza un sistema visual de tableros, listas y tarjetas para ayudar a organizar tareas y colaborar en equipo. Cada tablero de Trello representa un proyecto, y dentro de los tableros, las listas pueden usarse para representar diferentes etapas del flujo de trabajo. Las tarjetas representan tareas individuales que pueden moverse entre listas a medida que avanzan en el proceso. Trello se usará para llevar trazabilidad de las tareas realizadas, facilitando la planificación de las iteraciones y dejando constancia del progreso para luego escribir la memoria.

2.3.5. GitHub

GitHub [14] es una plataforma de alojamiento de código fuente y control de versiones basada en Git, que permite la colaboración en proyectos de software. GitHub se utilizará para almacenar el código del proyecto, gestionar el control de versiones y facilitar el despliegue continuo, asegurando la integridad y evolución del código fuente.

Estado del Arte

Para comprender correctamente el objetivo perseguido con este TFG, así como la problemática abordada, es necesario dar a conocer algunos conceptos referentes a la computación cuántica, al igual que el estado actual en que esta se encuentra. A continuación se exponen dichos conceptos, además de un estudio de algunos de los proveedores que conforman el mercado actual, y las soluciones que existen hoy día para lidiar con los desafíos que supone esta heterogeneidad en el mercado tecnológico.

3.1. Computación Cuántica

La computación cuántica es un campo multidisciplinario que combina aspectos de la informática, la física y las matemáticas, utilizando principios de la mecánica cuántica para resolver problemas complejos con mayor rapidez que las computadoras clásicas. Las computadoras cuánticas aprovechan fenómenos cuánticos como la superposición y la interferencia para afrontar ciertos problemas más eficientemente que las computadoras tradicionales. Áreas como el *machine learning*, la optimización y la simulación de sistemas físicos son ejemplos donde las computadoras cuánticas podrían ofrecer una ventaja significativa en velocidad, abordando problemas que actualmente están fuera del alcance incluso de las supercomputadoras más potentes [15].

Los ordenadores cuánticos usan *qubits*, que a diferencia de los bits clásicos, que sólo pueden ser 0 o 1, estos pueden existir en una superposición de estados gracias a los principios de superposición y entrelazamiento cuántico (figura 1). Esto permite que los *qubits* realicen múltiples cálculos simultáneamente, proporcionando un potencial de procesamiento exponencialmente mayor en comparación con los ordenadores clásicos para ciertos tipos de problemas.

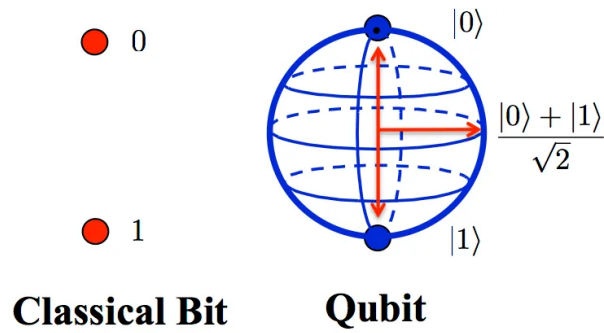


Figura 1: Comparación entre bit clásico y *qubit* [16]

Una de las principales arquitecturas de programación cuántica es la programación basada en circuitos cuánticos (ejemplo en figura 2). En este modelo, los algoritmos cuánticos se representan como circuitos cuánticos compuestos por *qubits* y puertas cuánticas que manipulan estos *qubits*. Las puertas cuánticas son análogas a las puertas lógicas en computación clásica, pero operan de manera probabilística y pueden afectar a múltiples *qubits* simultáneamente. Este enfoque se utiliza en la mayoría de las plataformas de hardware cuántico actuales y es fundamental para desarrollar y ejecutar algoritmos cuánticos que pueden, por ejemplo, factorizar números grandes, resolver problemas de optimización compleja, o simular interacciones moleculares de forma mucho más eficiente que los métodos tradicionales.

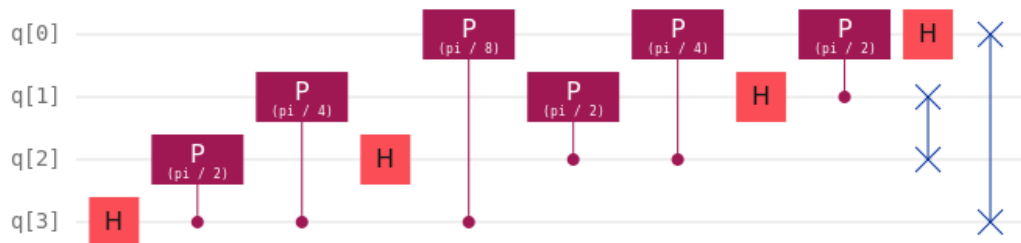


Figura 2: Transformada de Fourier Cuántica para 4 *qubits*

3.2. Situación y Problemática

La computación cuántica, a pesar de su gran potencial, enfrenta varios desafíos, siendo uno de los más significativos la heterogeneidad entre los proveedores de servicios cuánticos. Cada proveedor de servicios cuánticos, como Amazon Braket, IBM Quantum, IonQ y Rigetti, desarrolla y ofrece acceso a sus propias tecnologías y enfoques de hardware cuántico, que varían significativamente entre sí. Esta diversidad se traduce en diferencias en la representación y ejecución de algoritmos cuánticos, la medición de resultados, los lenguajes de programación utilizados y las interfaces de programación de aplicaciones (*APIs*).

Esta heterogeneidad crea varios problemas que suponen una traba para desarrolladores y empresas que quieran apostar por esta tecnología. Ante todo, hay una clara falta de estandarización que dificulta el desarrollo de software cuántico interoperable entre diferentes plataformas. Un algoritmo cuántico escrito para una plataforma puede requerir modificaciones sustanciales para ejecutarse en otra, lo que aumenta la complejidad y el tiempo de desarrollo [17].

Además, la divergencia en los métodos de acceso y en la estructura de costos entre los proveedores añade otra capa de complejidad, dificultando la tarea de desarrolladores para escoger una máquina cuántica que cumpla sus necesidades.

Esta es concretamente la problemática que busca abordar este TFG: mejorar la experiencia de usuario facilitando un punto en común donde consultar y acceder a distintas soluciones cuánticas.

3.3. Proveedores Cuánticos

Para indagar más en la problemática expuesta anteriormente, se ha realizado un estudio sobre los principales proveedores de servicios cuánticos, examinando en detalle qué servicios ofrecen y cómo se puede acceder a ellos.

A continuación, se presenta un análisis detallado producto de tal estudio del mercado. En él, se han considerado los proveedores Amazon Braket, IBM Quantum, IonQ y Rigetti; pues eran los estimados más relevantes para este proyecto.

3.3.1. Amazon Braket

Amazon Braket [18] es un servicio de computación cuántica en la nube ofrecido por Amazon Web Services (AWS). Braket provee un único punto de acceso a diferentes tecnologías cuánticas, entre las que se incluyen IonQ, QuEra y Rigetti. De este modo, ofrece una alternativa flexible y cómoda que permite a los usuarios diseñar, probar y ejecutar algoritmos cuánticos en múltiples plataformas de hardware cuántico sin necesidad de dedicar el esfuerzo mayor que supondría migrar entre diferentes proveedores, pues está todo integrado en Braket. Además, Amazon Braket ofrece una interfaz unificada (figura 3) que permite identificar de un vistazo general aquellos dispositivos que ofrece, destacando el estado de cada uno para conocer su disponibilidad.

Proveedor de hardware	Dispositivo	Estado	Descripción
Amazon Web Services	SV1	ONLINE	Simulador de vectores de estado de 34 cúbits de Amazon Braket
Amazon Web Services	TN1	ONLINE	Simulador de red de tensores de 50 cúbits de Amazon Braket
Amazon Web Services	DM1	ONLINE	Simulador de matriz de densidad de 17 cúbits de Amazon Braket
IonQ	Harmony	OFFLINE	QPU de trampa de iones con modelo de puerta universal de 11 cúbits
IonQ	Aria 1	OFFLINE	QPU de trampa de iones con modelo de puerta universal de 25 cúbits con compatibilidad con la mitigación de errores
IonQ	Aria 2	OFFLINE	QPU de trampa de iones con modelo de puerta universal de 25 cúbits compatible con la mitigación de errores
IonQ	Forte 1	SOLO BAJO RESERVA	QPU de trampa de iones con modelo de puerta universal de 36 cúbits que admite la mitigación de errores
IQM	Garnet	ONLINE	QPU superconductora con modelo de puerta universal de 20 cúbits
QuEra	Aquila	ONLINE	QPU analógica de 256 cúbits basada en átomos neutros
Rigetti	Aspen-M-3	OFFLINE	QPU superconductora con modelo de puerta universal de 79 cúbits
Rigetti	Ankaa-2	ONLINE	QPU superconductora con modelo de puerta universal de 84 cúbits

Figura 3: Dispositivos ofrecidos por Amazon Braket

En la figura anterior también se puede apreciar cómo Braket ofrece tres simuladores propios: SV1, TN1 y DM1; cada uno con características únicas que permiten al usuario simular la ejecución de sus circuitos antes de enviarlos a hardware cuántico real. Es más, Braket asegura una disponibilidad total a estos simuladores, por lo que nunca se encontrarán en mantenimiento y resultan ideales para realizar pruebas.

La flexibilidad de Braket y su integración con AWS hacen que sea una opción atractiva para empresas y desarrolladores que buscan experimentar con diferentes tecnologías cuánticas sin la necesidad de comprometerse con un solo proveedor de hardware.

En términos de precios, en Amazon Braket no se requiere de pagos anticipados, únicamente se paga por lo que se usa. Se basa en un sistema de acceso bajo demanda, y los precios de cada dispositivo pueden consultarse en el panel expuesto en la imagen 3 previa. No obstante, cuenta

también con un pequeño plan gratuito, muy beneficioso para desarrolladores principiantes que quieran comenzar a experimentar con circuitos sencillos. Este plan ofrece una hora gratuita de simulación de circuitos cuánticos al mes¹ para SV1, DM1, TN1, o cualquier combinación de ellos. Para el resto de dispositivos oferta una hora de cómputo total, que una vez agotada no es recargable gratuitamente, ya pasaría al plan bajo demanda.

Desde el punto de vista que plantea el proyecto, Amazon Braket será interpretado como un proveedor de terceros o intermediario, que ofrece sus propios simuladores junto con los sistemas de los proveedores asociados a él. Dado que carece de una API REST mediante la cual obtener información sobre sus dispositivos, la información de Braket y sus asociados se extraerá mediante el uso del SDK oficial de Python para AWS.

3.3.2. IBM Quantum

IBM Quantum [19] es uno de los pioneros en el campo de la computación cuántica y ofrece acceso a sus computadoras cuánticas superconductoras a través de *IBM Cloud*², donde oferta 12 QPUs distintas y 5 simuladores. IBM Quantum utiliza un enfoque basado en circuitos cuánticos y proporciona una suite de herramientas de software (SDK), Qiskit, que permite a los usuarios desarrollar y ejecutar programas cuánticos. Qiskit es una plataforma de código abierto que facilita el diseño, la simulación y la implementación de algoritmos cuánticos en los dispositivos de IBM.

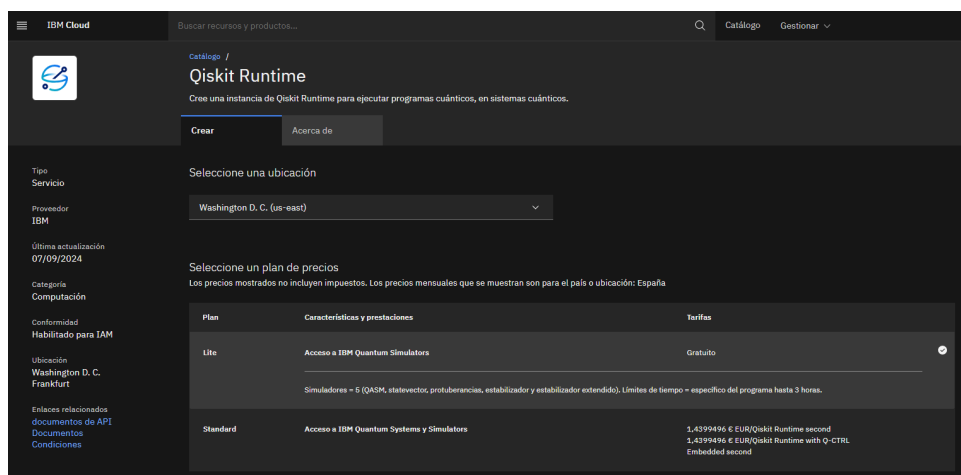


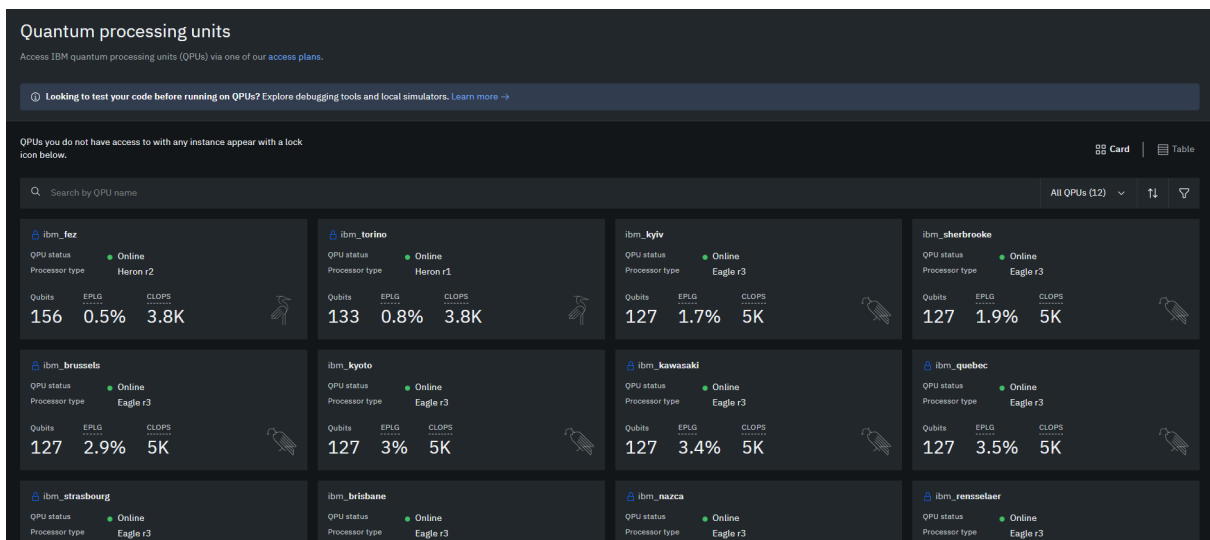
Figura 4: Captura del servicio *Qiskit Runtime*, a través de IBM Cloud

¹Únicamente durante los primeros doce meses de uso de Amazon Braket.

²<https://www.ibm.com/es-es/cloud>

IBM Quantum también proporciona acceso gratuito a algunos de sus dispositivos, concretamente el plan gratuito ofrece 10 minutos de cómputo al mes en cualquiera de las QPUs autorizadas en este plan³, lo que permite a estudiantes, investigadores y desarrolladores explorar la computación cuántica sin coste. Otras alternativas se pueden encontrar en su página de precios⁴.

Además, IBM se muestra muy comprometido con la educación y la creación de comunidad, ofreciendo numerosos recursos de formación y oportunidades de colaboración para fomentar el crecimiento del ecosistema cuántico. Entre más de las herramientas que provee IBM se encuentra *IBM Quantum Platform*⁵ (figura 5), una plataforma específica para la computación cuántica que además conecta directamente con *IBM Composer*, una interfaz visual e intuitiva para construir y ejecutar circuitos cuánticos.



Quantum processing units

Access IBM quantum processing units (QPUs) via one of our access plans.

Looking to test your code before running on QPUs? Explore debugging tools and local simulators. Learn more →

QPUs you do not have access to with any instance appear with a lock icon below.

Search by QPU name

All QPUs (12)

QPU name	QPU status	Processor type	Qubits	EPLG	CLOPS
ibm_fez	Online	Heron r2	156	0.5%	3.8K
ibm_torino	Online	Heron r1	133	0.8%	3.8K
ibm_kyiv	Online	Eagle r3	127	1.7%	5K
ibm_sherbrooke	Online	Eagle r3	127	1.9%	5K
ibm_brussels	Online	Eagle r3	127	2.9%	5K
ibm_kyoto	Online	Eagle r3	127	3%	5K
ibm_kawasaki	Online	Eagle r3	127	3.4%	5K
ibm_quebec	Online	Eagle r3	127	3.5%	5K
ibm_strasbourg	Online	Eagle r3			
ibm_brisbane	Online	Eagle r3			
ibm_nazca	Online	Eagle r3			
ibm_remselaar	Online	Eagle r3			

Figura 5: Captura de IBM Quantum Platform

Siguiendo el planteamiento del proyecto, IBM ha sido seleccionado como objeto de estudio y aplicación dado que posee una de las plataformas cuánticas más maduras y con mayor soporte en la industria. Concretamente, los datos referentes a la plataforma Quantum y los dispositivos que esta ofrece se obtendrán a través de la *Qiskit Runtime REST API*⁶.

³Estas son *kyiv*, *sherbrooke*, *kyoto* y *brisbane*.

⁴<https://www.ibm.com/quantum/pricing>

⁵<https://quantum.ibm.com/services/resources>

⁶<https://docs.quantum.ibm.com/api/runtime>

3.3.3. IonQ

IonQ [20] es un proveedor de computación cuántica que se especializa en la tecnología de *qubits* de trampa de iones. Los sistemas de IonQ utilizan iones atrapados con láseres para realizar operaciones cuánticas, una tecnología que ofrece alta fidelidad y tiempos de coherencia prolongados en comparación con otras tecnologías de *qubits*. Los dispositivos de IonQ están disponibles a través de la nube en plataformas como Amazon Braket y Microsoft Azure Quantum. Y aunque existe la posibilidad de registrarse en IonQ para tener acceso directo a ellos, este es bastante limitado. Únicamente es accesible su simulador de 29 *qubits*, el cual dispone de diferentes modelos de ruido para simular un entorno como si fuera hardware cuántico real.

Si se opta por el registro, IonQ permitirá el acceso a su consola cloud (figura 6), desde donde se pueden visualizar todos los sistemas disponibles y observar sus características y detalles de arquitectura. Sin embargo, como se mencionó anteriormente, únicamente existe posibilidad de envío de circuitos al simulador. Pese a ello, como ventaja adicional, no parece haber límite en la cantidad de tiempo de cómputo procesada en el simulador. Según la documentación⁷ de IonQ, el simulador es de acceso gratuito para todo aquel que se registre, pero en ningún momento habla de límite de cómputo o de tiempo.

Backends

QPUs

Simulators

Ideal Simulator

simulator

● Available

Show details

Simulated Hardware Noise Model ⓘ

Ideal

Qubits ⓘ

29

Avg. Time in Queue

0min

IonQ Aria 1 Noisy Quantum Simulator

simulator

● Available

Show details

Simulated Hardware Noise Model ⓘ

Aria 1

Qubits ⓘ

25

Avg. Time in Queue

0min

IonQ Harmony Noisy Quantum Simulator

simulator

● Available

Show details

Simulated Hardware Noise Model ⓘ

Harmony

Qubits ⓘ

11

Avg. Time in Queue

0min

Figura 6: Captura de dispositivos de IonQ

⁷<https://docs.ionq.com/>

Este último aspecto será de gran provecho para el desarrollo de este TFG, pues la integración de envíos de trabajos a IonQ permitirá que cualquiera con una cuenta y su API key correspondiente pueda beneficiarse de este aspecto.

En lo que respecta a la obtención de datos, dado que IonQ proporciona una de las API REST⁸ mejor documentadas en este mercado de proveedores cuánticos, y junto con el envío gratuito de trabajos, ha sido indudable su inclusión en el proyecto y el uso de esta API REST.

3.3.4. Rigetti

Rigetti Computing [21] es una empresa que desarrolla y opera computadoras cuánticas superconductoras y ofrece acceso a su hardware a través de su plataforma de terceros como pueden ser Amazon Braket o Microsoft Azure Quantum. Rigetti ha desarrollado su propio lenguaje de programación cuántica, *Quil*⁹, y un conjunto de herramientas de software que facilitan el desarrollo de algoritmos cuánticos y su integración con aplicaciones clásicas.

Rigetti no ofrece acceso directo a sus dispositivos cuánticos, ni siquiera a través de su SDK, por lo que se ha debatido mucho sobre su inclusión en este proyecto. A pesar de todo, finalmente se ha incorporado para experimentar con herramientas de *web scraping*, puesto que la información sobre sus sistemas solo se encuentra disponible en la página web del proveedor. Otra opción sería la de acceder mediante alguna plataforma intermediaria, caso que precisamente está cubierto al contar con Amazon Braket entre los proveedores seleccionados para el proyecto.

3.4. Soluciones y Alternativas Actuales

En el contexto de la computación cuántica y la problemática de la heterogeneidad entre proveedores, existen algunas soluciones y herramientas diseñadas para facilitar la interoperabilidad y el uso eficiente de múltiples plataformas cuánticas. La más destacable de ellas es *PennyLane*¹⁰, una librería de Python de código abierto que permite el desarrollo de algoritmos cuánticos híbridos que pueden ejecutarse en diferentes sistemas cuánticos y clásicos, proporcionando flexibilidad y compatibilidad cruzada entre plataformas. Inicialmente ofrece acceso

⁸<https://docs.ionq.com/api-reference/v0.3/introduction>

⁹<https://pyquil-docs.rigetti.com/en/stable/>

¹⁰<https://pennylane.ai/>

a 6 simuladores locales, pero mediante su sistema de *plugins*¹¹ permite integrar una amplia variedad de proveedores. Además, asegura poder ejecutar el mismo circuito cuántico en QPUs de distintos proveedores sin tener que alterarlo [22].

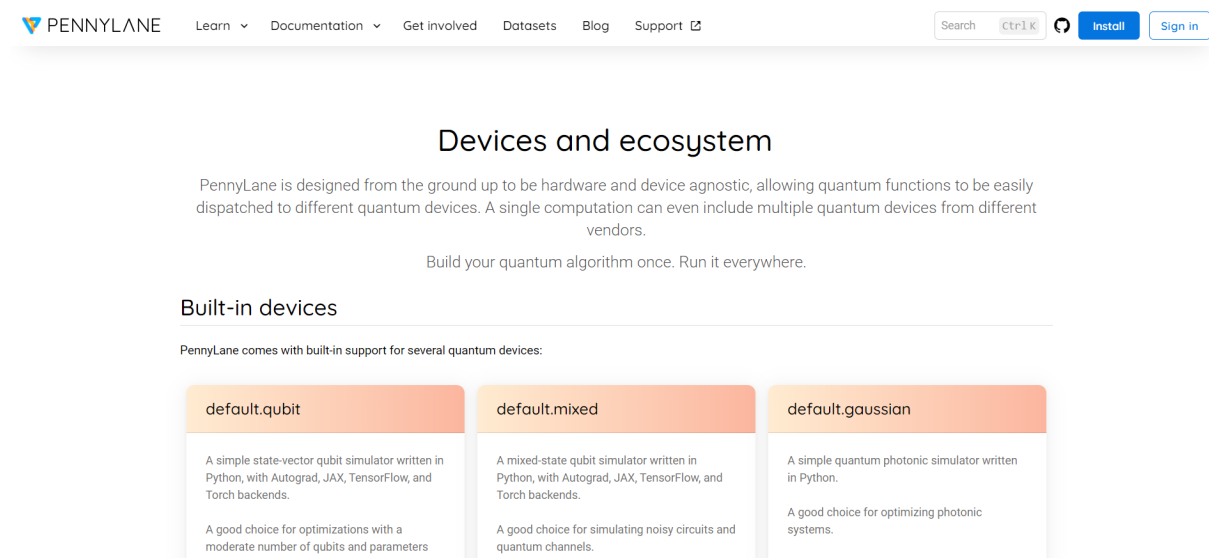


Figura 7: Captura de la página web de *PennyLane*

Con esto dicho, *PennyLane* supone una alternativa muy viable y prometedora, que a pesar de una falta de estandarización completa y compatibilidad universal, ofrece grandes avances en términos de crear un ecosistema más íntegro. No obstante, difiere de lo que se propone desarrollar con este TFG, lo cual se expone en el apartado a continuación.

3.5. Qué aporta *Quantum Proxy* en este contexto

La aplicación propuesta, bautizada como *Quantum Proxy*, es una plataforma web que busca abordar la heterogeneidad que existe entre los diferentes proveedores de ordenadores cuánticos. Su objetivo es proporcionar una mejor experiencia de usuario para quienes desean adentrarse en este campo, unificando diferentes servicios y proveedores en la medida de lo posible. Que a diferencia de *PennyLane* antes mencionada, que es un framework de Python, *Quantum Proxy* se presenta como una solución basada en la web, que no requiere instalaciones complicadas, lo que facilita el acceso a la misma.

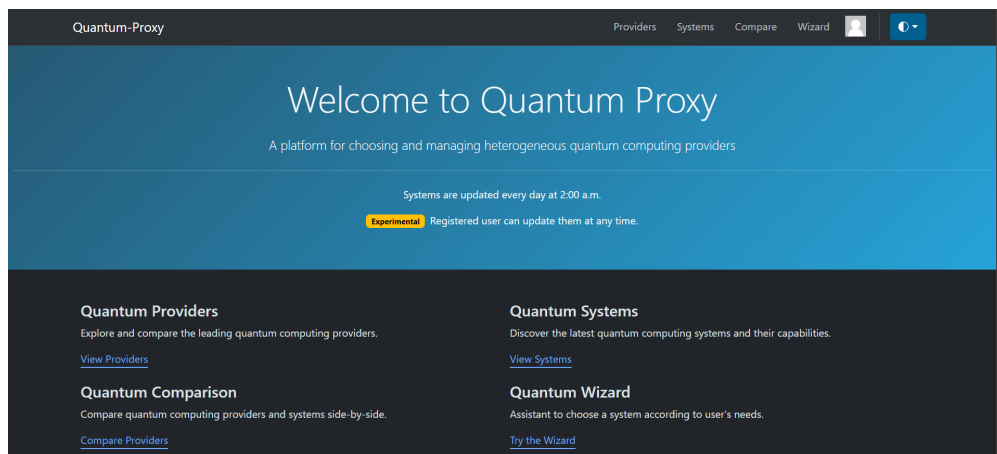
Quantum Proxy no pretende eliminar la heterogeneidad entre los proveedores ni crear un entorno de acceso común que resuelva las diferencias tecnológicas. En cambio, se enfoca en

¹¹<https://pennylane.ai/plugins/>

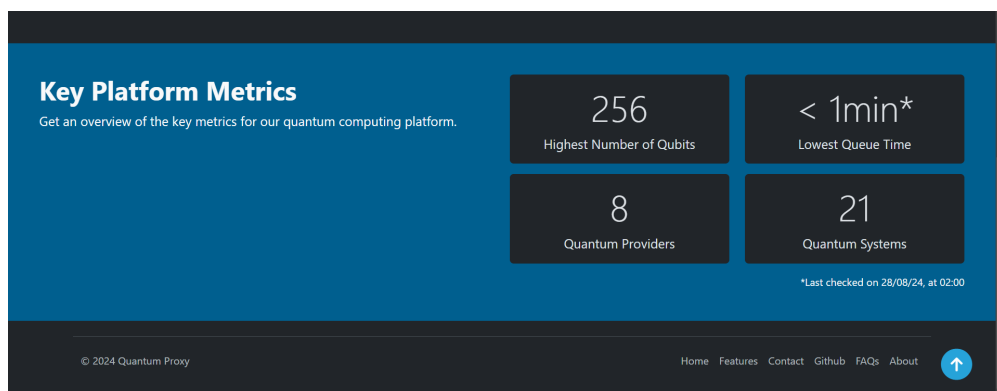
simplificar la interacción con este ecosistema diverso al ofrecer una plataforma unificada que actúa como un “proxy inverso” o “API Gateway cuántica”, permitiendo a los usuarios navegar entre diferentes opciones de proveedores con mayor facilidad.

La plataforma automatiza la recopilación y actualización de datos mediante un planificador, manteniendo la información de los proveedores siempre actualizada en la base de datos. Entre los servicios clave que ofrece *Quantum Proxy* se encuentran una herramienta de comparación entre proveedores y sus máquinas cuánticas, un asistente que ayuda al usuario a seleccionar un sistema cuántico adecuado a sus necesidades, y un panel unificado para enviar circuitos y ejecutarlos en hardware cuántico.

En resumen, *Quantum Proxy* se presenta como una solución que, sin intentar eliminar la diversidad existente, proporciona un punto de acceso centralizado que simplifica la navegación y búsqueda por el mejor ordenador cuántico que cumpla con los requerimientos del usuario.



(a) Página principal, fragmento 1



(b) Página principal, fragmento 2

Figura 8: Página principal de *Quantum Proxy*

4

Metodología de Trabajo

En el desarrollo de este TFG, se ha optado por una metodología ágil, caracterizada por su enfoque iterativo e incremental, lo que ha facilitado la adaptación continua ante cualquier imprevisto.

El proyecto se organizó en iteraciones, también conocidas como *sprints*, las cuales tenían una duración aproximada de 2 o 3 semanas. Al final de cada iteración, se realizaba una reunión con los tutores donde se exponía lo realizado durante esta, comentando los problemas encontrados y resolviendo las dudas que hubiesen podido surgir. La reunión finalizaba planificando el siguiente *sprint* y discutiendo los próximos pasos a seguir. Estas reuniones no solo permitieron mantener un flujo de trabajo organizado y enfocado, sino que también permitieron hacer ajustes y mejoras en tiempo real.

La planificación de cada iteración que constituye el TFG puede observarse visualmente en la imagen 9, representada mediante un diagrama de Gantt.

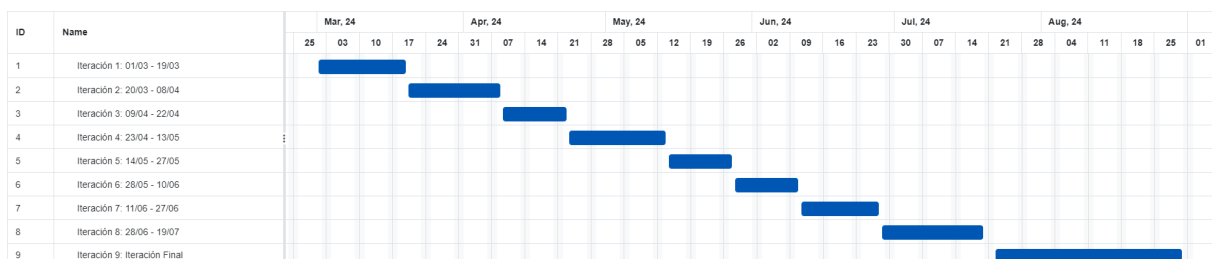


Figura 9: Diagrama de Gantt

4.1. Iteración 1 (Semanas 1-3)

La primera iteración se centró en organizar el proyecto, identificando los primeros pasos a tomar e investigando acerca de los principios de la computación cuántica. Entre otras cosas, estos fueron los primeros pasos:

- Para introducir y aprender las bases de la computación cuántica, fue de lectura esencial el libro *Programming Quantum Computers: Essential Algorithms and Code Samples* [23], por recomendación de los tutores.
- A su vez, para comprender la problemática que supone la diversidad de proveedores de computación cuántica, fue muy importante la lectura del artículo *Technological diversity of quantum computing providers: a comparative study and a proposal for API Gateway integration* [17].
- Se crea un proyecto en Trello para organizar las tareas siguiendo un enfoque *Scrum*¹², haciendo uso de listas como el *Backlog* y el *Sprint Backlog* para dar prioridad a lo que se desarrollaba en cada *sprint*.
- Se crea el repositorio GitHub para alojar el código y mantener un control de versiones del mismo.
- Comienza también la investigación de proveedores. Se indaga en el mercado de la computación cuántica cloud para conocer las distintas alternativas. Una vez identificados los proveedores a utilizar y cómo estos expresan sus datos, se trata de unificar atributos en común y no tan en común en un único modelo que sirva para tratarlos a todos.
- Al mismo tiempo, se comienza a experimentar con FastAPI y MongoDB, desarrollando algunos proyectos de prueba para comprender y entender estas tecnologías.
- Se propone crear cuenta *dummy* con acceso a claves, tokens, y API keys para obtener los datos necesarios. Puesto que la mayoría de proveedores no permiten consultar sus máquinas cuánticas si no se dispone de una cuenta.

¹²<https://scrumguides.org/>

- Se obtiene satisfactoriamente datos de Rigetti mediante el uso de web scraping con Selenium.

4.2. Iteración 2 (Semanas 4-6)

Tras el estudio y práctica con el framework FastAPI y los modelos de bases de datos NoSQL, da comienzo el diseño y desarrollo del backend:

- Se definen modelos `pydantic`¹³ para los proveedores, teniendo como base IonQ, IBM Quantum, y Rigetti. Para posteriormente integrarlos en el backend.
- Se añade una cuenta *dummy*. Un usuario creado única y exclusivamente para la realización de este TFG (`tfg.quantum.proxy@gmail.com`). Dicha cuenta será usada para autenticarse en distintos servicios de computación cuántica en la nube.
- Por otro lado, comienza a plantearse cómo estructurar los distintos módulos que tendrá la aplicación.

4.3. Iteración 3 (Semanas 7-8)

Durante esta iteración, se desarrolla en profundidad lo que comienza a ser el núcleo de la aplicación. Se implementan módulos esenciales para el funcionamiento de la misma.

- Comienza la investigación del *scheduler*. Un módulo que se encargará de actualizar los datos de proveedores y sistemas¹⁴ periódicamente.
- Quedan completamente definidas APIs para manejar datos de proveedores, sistemas, y usuarios de la plataforma.
- Se propone un sistema mediante el cual la definición de cómo extraer los datos de cada proveedor se halla en la base de datos, y no en el código. De esta manera, resulta más flexible la modificación de los proveedores, ya sea que surjan nuevos o desaparezcan otros.

¹³Pydantic es una librería de Python usada para la validación de datos

¹⁴“Sistemas” o incluso “*backends*” son términos que se usarán de ahora en adelante para hacer referencia a las máquinas cuánticas que ofrece un proveedor en concreto.

- Se definen distintos módulos para tratar con cada tipo de proveedor. Si se quiere añadir uno nuevo, no hay que modificar el código, solo añadir un nuevo proveedor cumpliendo con una serie de normas y reglas. Se hablará más en detalle de este proceso en la sección 6 sobre la implementación de los módulos.
- Se consolida por primera vez datos de los proveedores IonQ, IBM Quantum, y Rigetti. Una vez integrados correctamente en la plataforma, se plantea extraer datos también de Amazon Braket.
- Dadas las complicaciones con la API de Braket, resulta más sencillo hacer uso de su SDK.

4.4. Iteración 4 (Semanas 9-11)

En esta iteración continúa el desarrollo incremental del backend y de los servicios que este ofrece. Estos son algunos de los avances a destacar:

- Se implementa satisfactoriamente el *scheduler*, los datos son actualizados periódicamente a las 2:00 am. La recogida de información se paraleliza mediante threads, para no cortar el flujo de ejecución del hilo principal.
- Finalmente se adopta la idea de delegar a la base de datos la tarea de comprender cómo cada proveedor expone sus datos. Únicamente se realizan algunas modificaciones para mejorar su eficiencia.
- Amazon Braket es añadido como proveedor y se estudia la posibilidad de añadir Azure Quantum [24].

4.5. Iteración 5 (Semanas 12-13)

A partir de esta iteración se empieza a dedicar algo de tiempo al despliegue, puesto que el backend va tomando forma y pronto comenzarán las pruebas de despliegue. De nuevo, estos son algunos avances destacables:

- Dada la complejidad que supondría extraer los datos de máquinas de Azure, este se descarta y se deja como opción a desarrollo en un futuro por falta de tiempo (además, el plan gratuito era muy limitado y con muchas trabas).

- Con una buena base del backend desarrollada, se comienza a plantear su empaquetamiento en un `Dockerfile`. Al mismo tiempo, se empieza a escribir un fichero *docker compose* para su despliegue en local cuando todos los servicios tengan su `Dockerfile` definido.
- Por otro lado, cabe destacar que en este punto del proyecto se descartó la idea de realizar pruebas como se planteada en el anteproyecto. Se consideró que no tiene mucho sentido en un escenario como el que abarca este TFG. Esto es pues no es un programa al uso o parametrizado, se encarga de recoger datos periódicamente mediante APIs, *web scraping*, y SDKs; no hay “nada” sustancial que probar. Las únicas pruebas posibles (o validaciones) serían sobre el código, sobre cómo se comporta el backend (tests unitarios, y quizás de integración), pero se consideró que no era lo suficientemente importante para este proyecto y que debería dársele prioridad a otros aspectos.

4.6. Iteración 6 (Semanas 14-15)

En vista que el backend va bien encaminado, en esta iteración comenzó el desarrollo del frontend, finalizando con una primera versión del mismo desplegada en la nube. Estos son algunos de los progresos:

- Comienza el desarrollo del frontend, y con ello se plantea una primera versión de la web que ofrece un servicio de registro y autenticación de usuarios y cuentas al completo. Proporcionando entre otras cosas: autenticación mediante tokens JWT y refresh tokens, servicio de recuperación de contraseña, y verificación del email.
- Poco a poco se van creando las distintas interfaces de la plataforma, entre ellas la vista de proveedores y de sistemas.
- Puesto que no todo es automatizable, y no es posible obtener todos los datos que se quisiera de un proveedor, se proporciona un post-procesado en la recolección de datos que permite añadir información adicional.
- Finaliza la iteración con su primer despliegue por medio de Vercel y Render.

4.7. Iteración 7 (Semanas 16-18)

Durante la iteración número 7, el desarrollo continúa enfocado en el frontend, diseñando más vistas y funcionalidades que den bastante juego a la plataforma. Concretamente, se implementan las siguientes funcionalidades:

- Se implementa la comparativa de proveedores y máquinas cuánticas.
- Se implementan temas en la plataforma, ofreciendo un tema claro y otro oscuro en función del gusto del usuario final.
- Se mejora la interfaz del sistema de usuarios, permitiendo añadir API keys de forma sencilla e intuitiva. A su vez, se implementa la encriptación de las mismas para que en caso de filtración de la base de datos, no haya ningún peligro al no quedar estas expuestas.

4.8. Iteración 8 (Semanas 19-21)

En esta penúltima iteración se implementaron características clave para la aplicación, que junto con la finalización del fichero *docker compose* prácticamente ponían fin al desarrollo de más peso y prioridad.

- Se implementa una primera versión del *Wizard*. Primer prototipo del asistente, que solo contaba con su esqueleto para dar una idea de cómo se comportaría. La funcionalidad sería añadida en la siguiente iteración.
- Se implementa el envío de circuitos. Puesto que por falta de tiempo supondría mucho trabajo implementarlo para varios proveedores (dada la heterogeneidad entre ellos) se propone el envío de circuitos para el simulador de IonQ, que ofrece un plan gratuito bastante flexible para realizar las pruebas deseadas.
- Se termina el fichero *docker compose* para su despliegue en un entorno local.

4.9. Iteración Final (Semanas 22-27)

La última iteración, ya con todo el proyecto casi finalizado, se focalizó en pulir los últimos aspectos del frontend, y a desarrollar toda la documentación referente al desarrollo del TFG.

- Mejoras en la mayoría de las interfaces de la plataforma: página principal mejor estilada, vista de proveedores más completa, vista del *Wizard* mejor documentada, etc.
- Se provee de aspectos *responsive* a la mayor parte de la aplicación, para una experiencia cómoda y flexible independientemente del dispositivo desde el que se visite. Sin embargo, para potenciar al máximo el aprovechamiento de la plataforma, es recomendable su uso en un ordenador, y no en un dispositivo móvil.
- Se perfecciona el envío de trabajos. Dotando a los resultados de cada trabajo una vista amigable y con una tabla interactiva y agradable visualmente.
- Se escribe la memoria, junto con los manuales de usuario y de instalación.

5

Arquitectura y Modelado del Sistema

El diseño de la aplicación se ha basado en una arquitectura por capas que separa claramente los servicios que actúan en la aplicación. Esta arquitectura permite una mayor flexibilidad, escalabilidad y facilidad de mantenimiento, facilitando el desarrollo y la integración de nuevos servicios en el futuro. A continuación, se describe la arquitectura general del sistema y cómo interactúan los diferentes componentes, así como las clases implementadas y sus relaciones.

5.1. Visión General de la Arquitectura

Como se ha mencionado anteriormente, la plataforma está constituida siguiendo un enfoque de arquitectura de tres capas: frontend, backend y base de datos. Cada capa es responsable de manejar una parte específica del sistema y se comunica únicamente con su correspondiente directa a través de interfaces bien definidas. Este enfoque permite que cada capa se desarrolle, despliegue y mantenga de manera independiente, proporcionando una estructura clara y organizada para el proyecto.

- **Frontend:** La capa de frontend se encuentra en lo más alto en la jerarquía de servicios, es la única expuesta al exterior, la única accesible por el usuario. Está implementada utilizando Angular, un framework de desarrollo de aplicaciones web que permite construir interfaces de usuario dinámicas. Únicamente tiene acceso al backend.
- **Backend:** La capa de backend se encuentra en un punto intermedio en esta jerarquía, teniendo únicamente acceso a la base de datos. Está desarrollada utilizando FastAPI,

un framework moderno que permite construir APIs RESTful de alto rendimiento, proporcionando validación automática de datos y documentación interactiva. El backend maneja la lógica de negocio de la aplicación, gestionando las solicitudes del frontend.

- **Base de Datos:** La base de datos utilizada es MongoDB, una base de datos NoSQL orientada a documentos que almacena datos en formato JSON. MongoDB es ideal para manejar datos no estructurados o semiestructurados, lo que será clave para el modelo requerido para este proyecto puesto que cada proveedor gestiona sus datos de manera diferente.

A continuación se presenta un diagrama que representa la arquitectura antes descrita. En el mismo esquema se incluyen dos enfoques distintos de despliegue: en un entorno local con *Docker Compose*, y en la nube con plataformas de despliegue. Ambos serán descritos posteriormente en la sección 8.

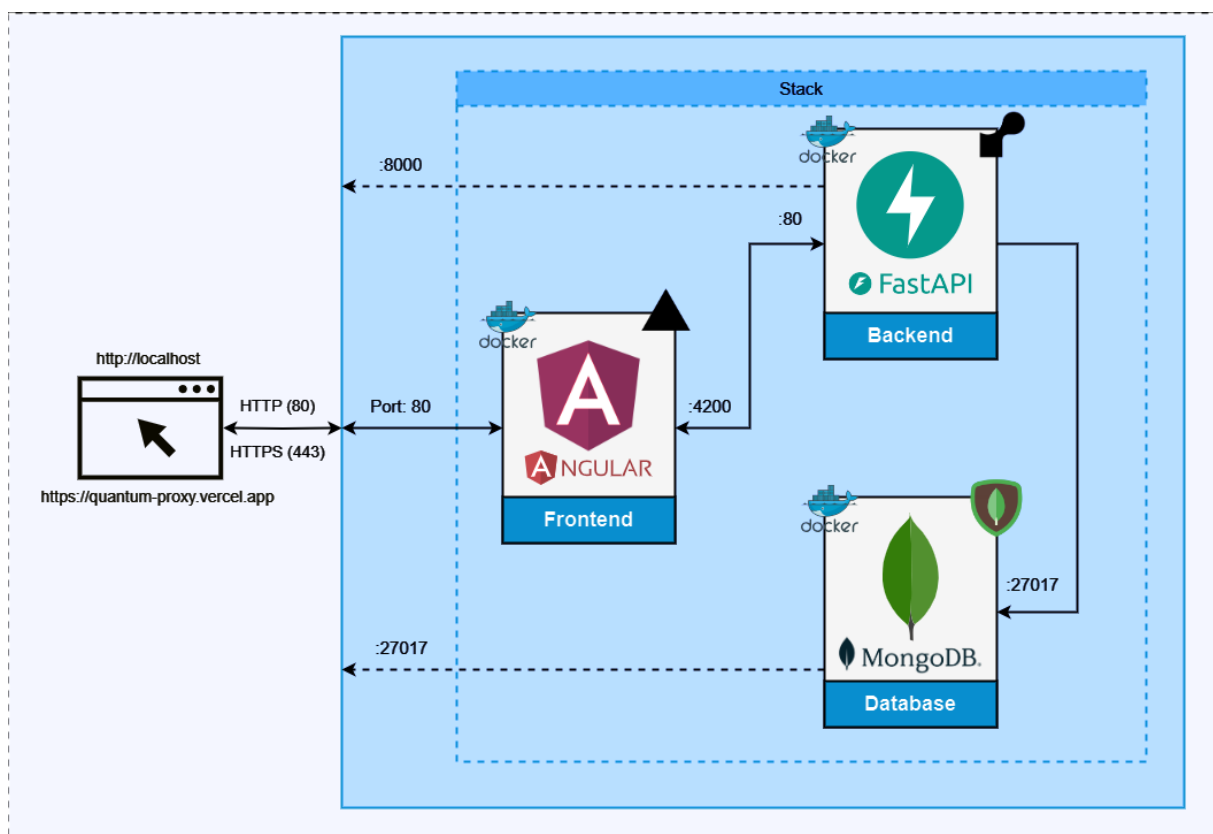


Figura 10: Diagrama de infraestructura del sistema

5.2. Arquitectura del Backend

Si se presta atención al código del proyecto¹⁵, se puede observar cómo el backend se compone esencialmente de cinco carpetas que ofrecen distintas funcionalidades:

- **La carpeta *database*:** sustenta el código relativo a la base de datos, esto es: conexión con la misma, métodos de acceso a ella, modelos que la componen, y esquemas que proveen equivalencias entre modelos.
- **La carpeta *modules*:** aloja los distintos módulos que hacen posible la obtención de los datos ofrecidos por cada proveedor. Estos podrán ser obtenidos por uno de los siguientes métodos: consultas API, consultas a través de un SDK, extracción por *web scraping*. El método de consulta variará en función de las características del proveedor, y se detallará el funcionamiento de este sistema de módulos en la sección 6.
- **La carpeta *routers*:** corresponde al código que implementa los distintos *endpoints* de la API REST. Gracias a ellos es posible construir una lógica consistente para interactuar con proveedores, máquinas cuánticas, tareas de circuitos cuánticos, y cuentas de usuario.
- **Las carpetas *security* y *utils*:** aunque sean dos carpetas separadas, ambas contribuyen al mismo objetivo de brindar apoyo y seguridad al resto de módulos que componen la aplicación. En su interior se encuentran servicios como: el cifrado de los datos sensibles, el servicio de emails automatizados, y el más importante, el planificador (o *scheduler*) que se encarga de actualizar diariamente los datos que se almacenan.

5.3. Arquitectura del Frontend

El frontend de la aplicación está construido con Angular, que utiliza un enfoque basado en componentes. De este modo, cada parte de la interfaz de usuario se divide en componentes reutilizables que manejan su propia lógica y presentación, dando una mayor flexibilidad al código.

Los componentes de Angular se comunican con el backend a través de servicios HTTP, utilizando el módulo HttpClient de Angular para enviar y recibir datos. Esta comunicación es

¹⁵<https://github.com/fcristallocagnoli/Quantum-Proxy>

asíncrona, lo que permite que la aplicación responda de manera eficiente a las acciones del usuario sin bloquear la interfaz de usuario.

A continuación se definen los componentes desarrollados para la aplicación, siguiendo la estructura de carpetas del proyecto en GitHub¹⁶

- `_components/`: es una carpeta que reúne varios componentes sencillos. Entre ellos se encuentran el sistema de gestión de alertas, el gestor de temas, y el componente de actualización de API keys.
- `_helpers/`: define configuraciones varias para el proyecto, así como interceptores de peticiones HTTP.
- `_models/`: contiene las clases e interfaces que requiere el frontend.
- `_services/`: define los servicios de la aplicación. En el contexto de Angular, son los encargados de definir las peticiones HTTP que se realizarán al backend.
- `account/`: componente que estructura las vistas relacionadas con cuentas de usuario. Entre otras, incluye el login, registro, y restablecimiento de contraseña.
- `admin/`: contiene dos componentes en su interior, ambos referentes a la administración de la plataforma. El primero gestiona las cuentas de usuario, mientras que el segundo permite gestionar aspectos de la base de datos.
- `comparative/`: define el servicio de comparativa, el cual permite comparar dos proveedores o dos sistemas entre sí.
- `home/`: aloja la página principal de la plataforma.
- `jobs/`: incluye todo lo relacionado con trabajos. Esto es, crear nuevos trabajos y consultar sus resultados.
- `profile/`: relativo al perfil del usuario.
- `providers/`: incluye las vistas de los proveedores.
- `systems/`: incluye las vistas de los sistemas (ordenadores cuánticos).

¹⁶<https://github.com/fcristallocalnoli/Quantum-Proxy>

- `wizard/`: contiene el servicio de asistente para la elección de un ordenador cuántico.

5.4. Modelo de Datos y Estructura de la Base de Datos

La base de datos MongoDB se utiliza para almacenar todos los datos relacionados con la aplicación, incluyendo la información del usuario, los proveedores designados, y sus correspondientes ordenadores cuánticos. Como ya se ha manifestado, MongoDB es particularmente adecuado para este proyecto debido a su capacidad para manejar datos no estructurados, sin embargo, se definen una serie de modelos básicos para que tanto backend como frontend puedan tratar con los datos de la forma más estructurada posible.

5.4.1. Modelos para la gestión de usuarios

- La clase *UserModel* representa un usuario en su estado más básico. Conteniendo información personal del usuario, así como roles y API keys asociadas. Hereda de la clase *BaseModel*, una clase proporcionada por `pydantic` para crear modelos.

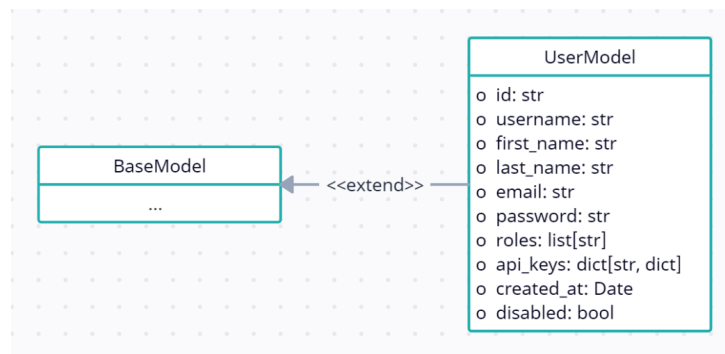


Figura 11: UserModel — Modelo pydantic

- La clase *UserInDBModel* (figura 12) representa a un usuario almacenado en la base de datos. Contiene atributos extra como tokens para la autenticación del usuario. Hereda por tanto de la clase *UserModel* vista antes.

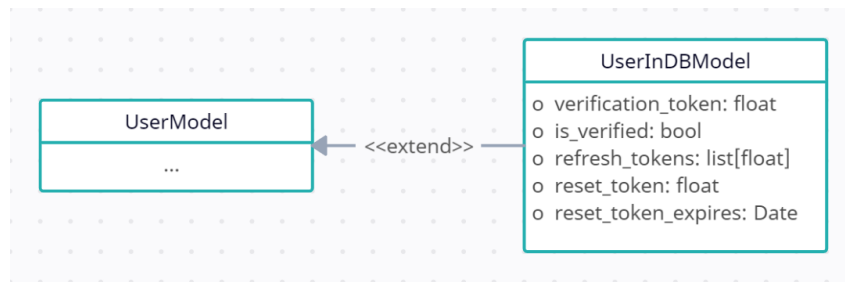


Figura 12: UserInDBModel — Modelo pydantic

5.4.2. Modelos para la gestión de proveedores

La gestión de proveedores se vuelve algo más compleja, por lo que requerirá de varias clases auxiliares que contribuyen a un modelo común. La clase principal es *BaseProviderModel*, la cual será capaz de albergar cualquier tipo de proveedor.

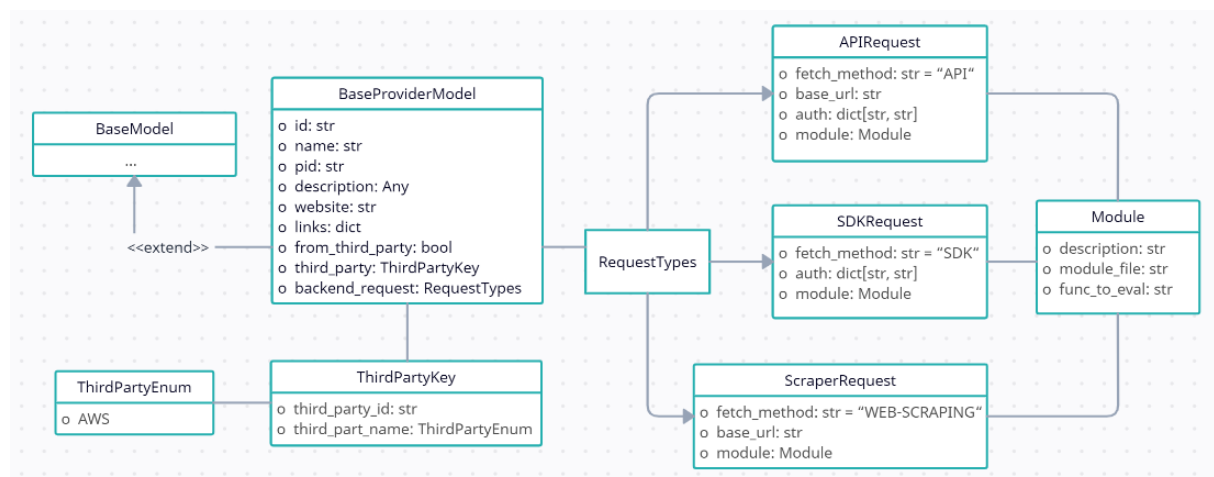


Figura 13: BaseProviderModel junto con clases auxiliares — Modelo pydantic

Como podemos apreciar en la figura 13, aparte de heredar de la clase *BaseModel*, el modelo de proveedor tiene una conexión con distintas clases auxiliares que servirán para identificar el tipo de proveedor y cómo se extraerán sus datos. A continuación se detallan dichas clases:

- *BaseProviderModel* cuenta con atributos sencillos como el nombre, un pid (*Provider Identifier*), o la descripción del mismo.

- Los campos *from_third_party* y *third_party* son los encargados de identificar qué tipo de proveedor de tercero¹⁷ está modelando¹⁸.
- Por último, *backend_request* es probablemente el atributo más importante del modelo. Es el encargado de definir cómo se extraerán los datos de este proveedor. Los valores permitidos son los siguientes:
 - *APIRequest*: define la URL base para hacer las consultas API, autenticación si fuese necesario, y asocia una clase *Module*, que se detallará en breve.
 - *SDKRequest*: define la autenticación necesaria y asocia una clase *Module*.
 - *ScraperRequest*: define la URL base para la extracción con *web scraping* y asocia una clase *Module*.

La clase *Module* antes mencionada es algo especial, pues resulta fundamental para el funcionamiento de la aplicación. Para comprenderla, se debe mencionar que inicialmente se planteó la idea de almacenar el código responsable de extraer datos de cada proveedor en la base de datos. De este modo, la tarea de añadir o retirar proveedores quedaría desacoplada del código principal.

Sin embargo, esta idea acabó siendo descartada, pues a la hora de desarrollar el código específico para cada proveedor resultaba muy ineficiente trabajar directamente inyectándolo en la base de datos. Aún con la idea de delegar a la base de datos la función de agregar o eliminar proveedores, se planteó otra opción alternativa.

La alternativa propuesta fue, puesto que resulta poco eficaz almacenar el código directamente en la base de datos, almacenemos el código en el repositorio, junto al resto de la aplicación en una carpeta aislada `modules/source_files/`, y mediante la clase *Module* se indica desde la base de datos **qué fichero almacena el código a ejecutar** para el proveedor que corresponda, así como la **función principal** a ejecutar que devolverá con seguridad la lista de ordenadores cuánticos ofrecidos.

¹⁷Se definirá *proveedor de tercero* a aquel que actúe como intermediario para acceder a otros proveedores. Este es el caso de Amazon Braket.

¹⁸Actualmente únicamente existe soporte para Amazon Braket como proveedor de tercero, pero podría añadirse Microsoft Azure Quantum siguiendo este modelo.

De este modo, no solo se logra delegar la gestión de proveedores a la base de datos, sino que además se consigue aplicando el principio abierto-cerrado¹⁹ (*OCP*), pues permite que el sistema sea fácilmente extensible sin la necesidad de modificar el código original.

5.4.3. Modelos para la gestión de sistemas

Para manejar distintos tipos de ordenadores cuánticos, se ha ideado un modelo de clases donde cada sistema heredará de una base común, y luego se especializará añadiendo atributos propios de cada proveedor.

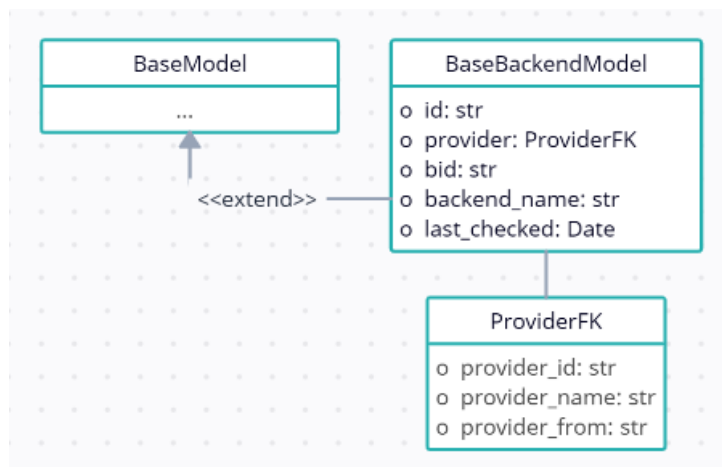


Figura 14: BaseBackendModel — Modelo pydantic

En la figura 14 se puede apreciar el modelo base antes mencionado, el cual incluye atributos como: el bid (*Backend IDentifier*), el nombre del ordenador cuántico, y la fecha de última actualización. Junto a ellos tiene asociado también un campo *ProviderFK*, el cual no es más que una referencia a una instancia de un proveedor (visto en 13).

ProviderFK contiene a su vez tres campos: el identificador del proveedor (generado por la base de datos), el nombre del proveedor, y un atributo *provider_from*, que cobra sentido cuando el proveedor se está obteniendo a través de un servicio de terceros como puede ser Amazon Braket.

¹⁹<https://www.arquitecturajava.com/el-principio-ocp-explicado-de-forma-sencilla/>

Visto el modelo base que comparten todas las máquinas cuánticas, a continuación se detallan uno a uno los modelos específicos para sistemas de cada tipo de proveedor.

5.4.3.1 IonQBackend Model

Se introduce en primer lugar el modelo para los ordenadores cuánticos de IonQ. En la figura 15 se puede observar los distintos campos extra que posee.

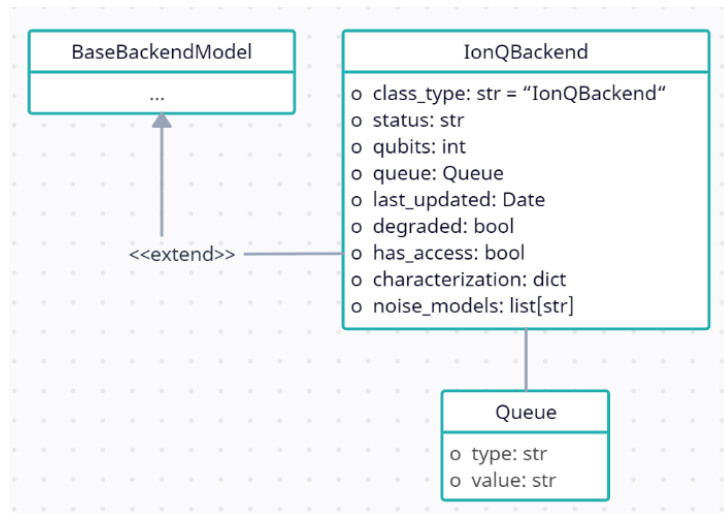


Figura 15: IonQBackendModel — Modelo pydantic

Como ya se introdujo, hereda de *BaseBackendModel*, e implementan una serie de atributos nuevos para este tipo de sistemas: el estado de la máquina, el número de *qubits*, la fecha de última actualización por parte del proveedor, información de la cola de trabajos, caracterización de la máquina y modelos de ruido.

A destacar, cabe mencionar que la clase *Queue* contiene dos atributos, *type* y *value*, pues para dar soporte a distintos tipos de cola se ha considerado así.

5.4.3.2 IBMBackend Model

Del modelo implementado para IBM Quantum, presente en la figura 16, se pueden destacar los siguientes atributos: el estado de la máquina, el número de *qubits*, información de la cola de trabajos, la fecha de última actualización por parte del proveedor, puertas soportadas y máximo número de experimentos y de ejecuciones.

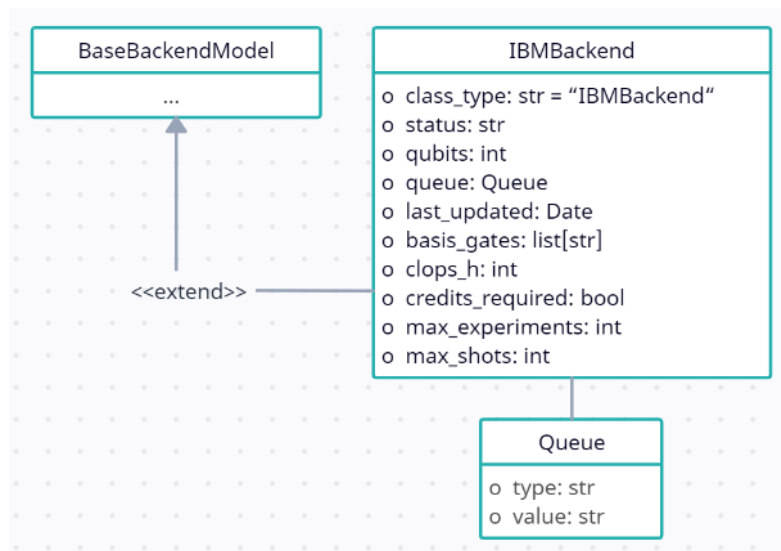


Figura 16: IBMBackendModel — Modelo pydantic

5.4.3.3 RigettiBackend Model

Del modelo de Rigetti hay poco que decir. Puesto que su información se extrae mediante *web scraping*, la cantidad de datos que se pueden obtener de su página web es muy limitada.

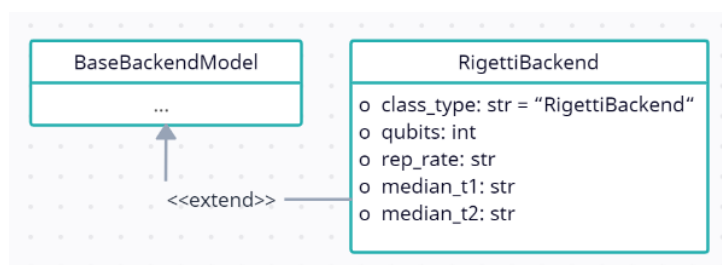


Figura 17: RigettiBackendModel — Modelo pydantic

5.4.3.4 AmazonBraketBackend Model

Observando la figura 18, destacamos los siguientes atributos: el estado de la máquina, el número de *qubits*, información de la cola de trabajos, puertas soportadas, rango de ejecuciones y precio del dispositivo.

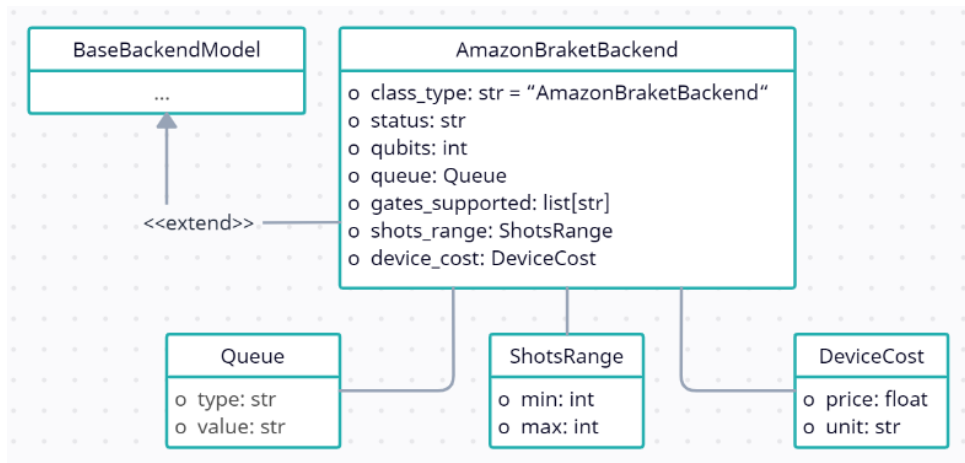


Figura 18: AmazonBraketBackendModel — Modelo pydantic

6

Diseño y Organización en Módulos

El diseño modular de la aplicación permite una estructura organizada y flexible, facilitando el desarrollo, el mantenimiento y la ampliación de la plataforma. A continuación, se describen los módulos principales que intervienen en el proceso de la toma de datos.

En el camino, se irá detallando el proceso de ejecución que sigue la aplicación para obtener, procesar, y almacenar todos los datos necesarios.

6.1. Visión general del Backend

Para ilustrar el funcionamiento del backend, se presenta el siguiente diagrama *BPMN* simplificado. En él se describe el proceso que ocurre una vez arranca la aplicación.

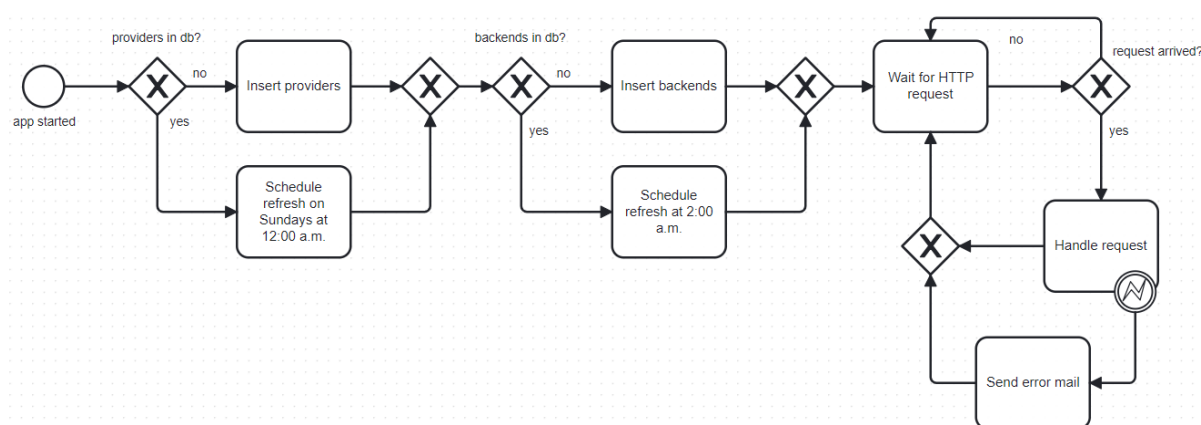


Figura 19: Diagrama BPMN del backend

El flujo es sencillo, una vez se inicia la aplicación, el primer paso es comprobar si ya hay datos almacenados relativos a proveedores o máquinas cuánticas. Si alguna de las colecciones de datos está vacías, se insertan los datos correspondientes, en caso contrario, se programa una actualización: diariamente a las 2:00 AM (UTC) para los ordenadores cuánticos, y semanalmente a las 12:00 AM (UTC) para los proveedores.

Una vez inicializada con datos, queda a la espera de peticiones HTTP por parte del usuario (desde la interfaz web). Si por algún motivo algo fallase, el sistema se encargará de notificar al administrador del sistema por medio del servicio de emails.

Los módulos implícitos que entran en juego en el proceso son:

- **Módulo del *scheduler***: contiene funciones específicas para organizar la obtención de datos. Actúa desde el momento que arranca la aplicación, y se queda corriendo en segundo plano. Se comunica con el módulo de *fetching*.
- **Módulo de *fetching***: dada su complejidad, se dedicará el apartado 6.2 a su explicación. Se encarga de obtener los datos de un proveedor, independientemente del tipo, siguiendo las directrices marcadas por la base de datos.
- **Módulo de *jobs***²⁰: será el encargado de manejar la creación de trabajos y su posterior envío a ordenadores cuánticos.
- **Módulo de peticiones HTTP**: fundamentalmente contiene la definición de los *end-points* de la API REST que actúa como intermediaria entre frontend y backend.

6.2. Módulo para la recopilación de datos

6.2.1. Integración con la base de datos

Si nos remontamos a la definición del modelo *BaseProviderModel* visto en el apartado 5.4.2, figura 13, se recalca la importancia que tenía el atributo *backend_request* y su componente *Module*. Esto es debido a que en ellos reside toda la lógica para obtener los sistemas de cada proveedor. Veámoslo en detalle con el ejemplo de IonQ a continuación.

²⁰Hay que destacar que dada la complejidad de configurar el envío de trabajos, y sumado a la falta de tiempo, únicamente se ha implementado el envío de trabajos al simulador de IonQ.

```

"backend_request": {
  "fetch_method": "API",
  "base_url": "http://api.ionq.co/v0.3",
  "auth": {
    "Authorization": "apiKey TOKEN"
  },
  "module": {
    "func_to_eval": "get_backends",
    "module_file": "ionq_api_code"
  }
}

```

Source Code 1: Atributo `backend_request` de IonQ

Siempre con el diagrama 13 en mente, se interpreta de la siguiente manera:

- *fetch_method* es igual a “API”, por lo que indica que los datos se proveerán a través de una API REST.
- *base_url* define la URL base para las consultas HTTP a la API.
- *auth* define la autorización necesaria para acceder a los datos. En este caso, bastará con incluir una cabecera “Authorization: apiKey TOKEN, donde *TOKEN* será sustituido por la clave presente en la cuenta *dummy*.
- *module* define el fichero que contiene el código para recoger los datos, junto con la función que se debe ejecutar para ello.

Otro ejemplo es el *backend_request* de Rigetti, donde los datos se obtienen mediante *web scraping* y se provee la URL base y los datos relativos al módulo:

```

"backend_request": {
  "fetch_method": "WEB-SCRAPING",
  "base_url": "https://qcs.rigetti.com/qpus",
  "module": {
    "func_to_eval": "get_backends",

```

```

        "module_file": "rigetti_ws_code_v1"
    }
}

```

Source Code 2: Atributo backend_request de Rigetti

O incluso Amazon Braket, donde se obtienen mediante el uso del SDK correspondiente, y el método de autenticación son unas variables de entorno requeridas para su correcto funcionamiento:

```

"backend_request": {
    "fetch_method": "SDK",
    "auth": {
        "env_vars": [
            "AWS_ACCESS_KEY_ID",
            "AWS_SECRET_ACCESS_KEY",
            "AWS_REGION"
        ]
    },
    "module": {
        "func_to_eval": "get_backends",
        "module_file": "braket_sdk_code"
    }
}

```

Source Code 3: Atributo backend_request de Amazon Braket

6.2.2. Aplicación directa en el código

Una vez se ha comprendido cómo el modelo almacenado en la base de datos dicta el comportamiento de la recopilación de datos, pasamos a introducir cómo se integra esto realmente con el backend.

En la figura 20 a continuación se puede apreciar todo el código relativo al módulo de *fetching*, que se encuentra dentro de la carpeta `modules/`. La cual a su vez, contiene la carpeta `source_files/`, que aloja los *scripts* individuales para cada proveedor.

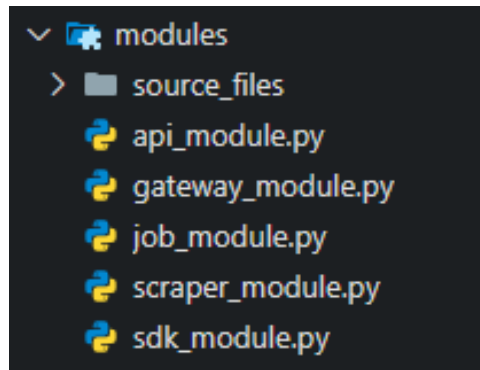


Figura 20: Modules — estructura de carpetas

El funcionamiento del sistema es el siguiente:

- Cada vez que el planificador mande ejecutar la actualización de los datos, llamará a la función `fetch_data` presente en 21, la cual se encuentra en el módulo `gateway_module.py`.
- Esta función ejecutará el módulo correspondiente, según se trate de una consulta via API, SDK, o extracción mediante *web scraping* (usando Selenium).
- Y finalmente, se devolverán los datos obtenidos en una versión normalizada.

```
def fetch_data(provider: BaseProviderModel):  
    request = provider.backend_request  
    if not request:  
        return None  
    match request.fetch_method:  
        case "API":  
            data = fetch_from_api(provider)  
        case "SDK":  
            data = fetch_from_sdk(provider)  
        case "WEB-SCRAPING":  
            data = fetch_from_ws(provider)  
        case _:  
            return None  
    return list(map(normalize_backend, data))
```

Figura 21: Función `fetch_data`

Internamente el funcionamiento desglosado por módulo es el siguiente:

■ Función `fetch_from_api`:

- Se identifica el fichero ubicado en `source_files` a ejecutar gracias a los atributos `module_file` y `func_to_eval` del atributo `module` en el proveedor.
- Si existe el atributo `auth`, se encargará de recopilar las API keys correspondientes desde la cuenta `dummy`.
- Por último, se ejecuta la función indicada por `func_to_eval` para obtener los datos.

■ Función `fetch_from_sdk`:

- Al igual que `fetch_from_api`, se identifica el fichero a ejecutar gracias a los atributos `module_file` y `func_to_eval` del atributo `module`.
- Similar al funcionamiento de `fetch_from_api`, si existe el atributo `auth`, se encargará de recopilar las API keys correspondientes desde la cuenta `dummy`. La diferencia es que para usar el SDK se requerirán variables de entorno, no tokens o API keys.
- Finaliza ejecutando la función indicada por `func_to_eval` para obtener los datos.

■ Función `fetch_from_ws`:

- Al igual que los anteriores, se identifica el fichero a ejecutar gracias a los atributos `module_file` y `func_to_eval` del atributo `module`.
- Se inicializa un webdriver de Chrome para proceder con el *web scraping*.
- Finaliza ejecutando la función indicada por `func_to_eval` para obtener los datos, en el entorno creado bajo el driver.

6.3. Recopilación: módulos del sistema

Para finalizar esta sección, se presentan una serie de diagramas que ilustran todos los módulos principales que componen la aplicación y cómo estos interactúan entre sí. Como el diagrama es bastante grande en tamaño, se dividirán según la capa a la que pertenezcan.

6.3.1. Módulos del Frontend

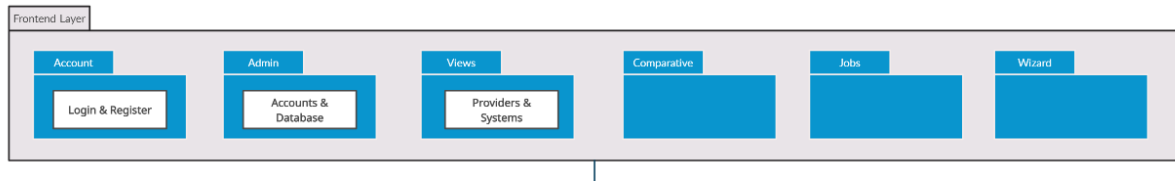


Figura 22: Módulos principales del Frontend

Se destacan los correspondientes a la gestión de cuentas, administración, vistas básicas, servicio de comparativa, de tareas y *wizard* o asistente. Esta hace uso de su inmediata inferior, la capa del *backend*.

6.3.2. Módulos del Backend

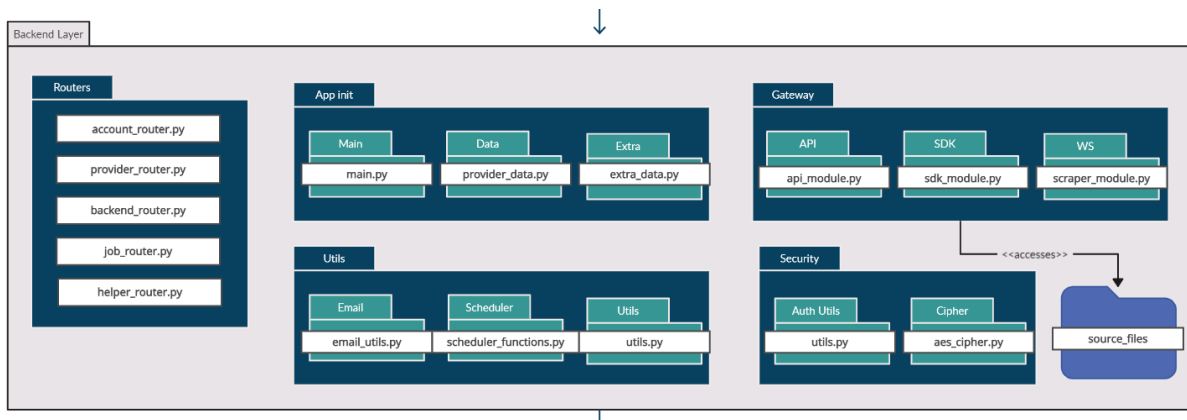


Figura 23: Módulos principales del Backend

En la imagen anterior figuran:

- Todos los *routers* que definen los *endpoints* de la aplicación y su comportamiento.
- Los módulos que entran en juego al inicializar la aplicación.
- Los módulos dentro de *Gateway* corresponden a aquellos encargados de obtener los datos de proveedores. Que a su vez, acceden a la carpeta *source_files*.
- Dentro de *utils* se encuentran servicios como el planificador o el envío de emails automatizado.

- En *Security* se encuentran los módulos de autenticación y cifrado de datos.

6.3.3. Módulos de la Base de Datos

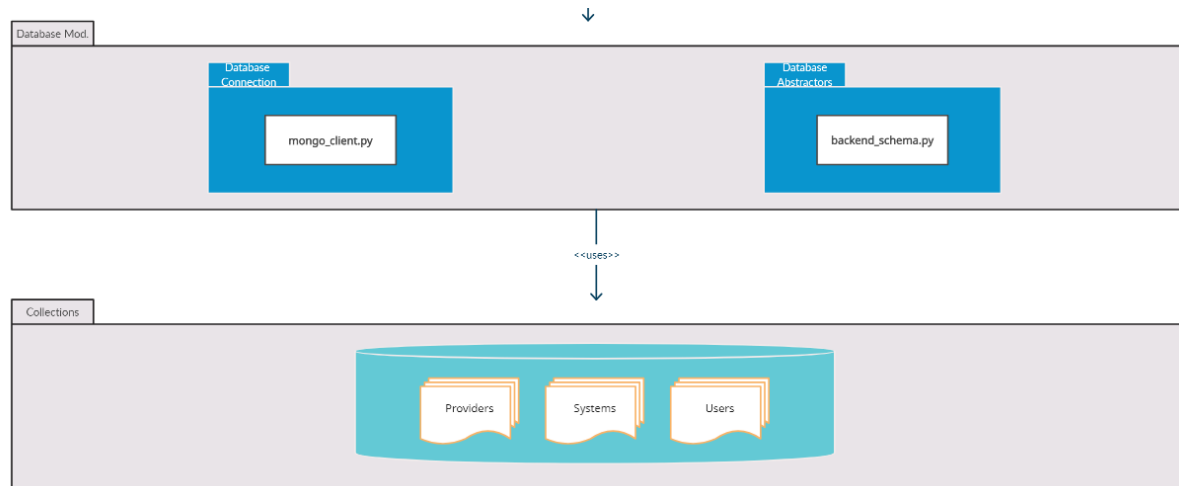


Figura 24: Módulos con relación a la Base de Datos

Por último, en relación con la base de datos se definen únicamente dos módulos:

- *mongo_client*: encargado de gestionar la conexión con la base de datos, así como los métodos para acceder a ella y modificarla.
- *backend_schema*: encargado de proporcionar una correspondencia entre los elementos almacenados y lo que luego procesará el backend. También aplica en viceversa.

A su vez, la base de datos se compone de tres colecciones: una para almacenar los proveedores, otra para las máquinas cuánticas, y otra para las cuentas de usuario.

Extensibilidad

En esta sección se pretende constatar la flexibilidad y facilidad que ofrece el sistema al momento de añadir nuevos proveedores a la plataforma. Esto es posible gracias al enfoque y aplicación del principio abierto-cerrado (*Open-Closed Principle*) que se ha dado a la aplicación.

7.1. Requisitos para Extender el Sistema

Antes de proseguir, es necesario entender que para añadir nuevos proveedores a la plataforma estos han de cumplir una serie de normas y reglas. De otro modo, la integración no sería posible.

El único requisito fundamental es que el proveedor debe ofrecer alguna vía para obtener información acerca de él, así como de los servicios y ordenadores cuánticos que oferte. Actualmente el sistema soporta tres métodos para obtener los datos: a través de consultas HTTP por medio de una API REST, a través de código configurable por medio de un SDK, o bien, si no existen estas alternativas, es posible configurar un *script* de *web scraping* que obtenga información basada en alguna página web.

El resto de normas a seguir se definen en el siguiente apartado 7.2, pues son relativos a cómo se configura su información en la base de datos.

7.2. Pasos para Extender el Sistema

A pesar de que el sistema se encuentra abierto a la extensión, y se ha procurado mantenerlo cerrado para su modificación, no libra de la tarea de modificar el código. Sí, el modelo propuesto aporta mucha flexibilidad, pero en el transcurso del proyecto se ha ido descubriendo que hay muchos aspectos que no son fáciles de abstraer, ni mucho menos automatizables.

Por ello, una vez que se tiene la certeza de que el nuevo proveedor a incluir es compatible

con alguno de los tres métodos descritos anteriormente, se puede proceder a su configuración en la base de datos. Los pasos se describen a continuación.

7.2.1. Configuración en la Base de Datos

El proveedor será configurado como un objeto *JSON* y se añadirá a la colección *providers*. Esto puede hacerse vía una conexión directa con la base de datos (usando *MongoDB Compass* o *mongosh*), o bien a través de la interfaz habilitada en la plataforma (véase la sección [A.5.3](#) del manual de usuario, figura [39](#)).

Tal objeto tendrá por atributos los descritos a continuación. Si no se especifica explícitamente, todo atributo se considera obligatorio.

- **name:** Nombre del proveedor.
- **pid:** Identificador del proveedor. Tiene la forma “native.<name>”, donde “native” indicará al sistema que a dicho proveedor se accede de forma directa.
- **description:** Opcional, indica una descripción para el proveedor.
- **website:** Opcional, pero recomendado, indica la página web del proveedor.
- **from_third_party:** Con posibles valores `true` o `false`, indica al sistema que el proveedor no proviene de un tercero. Servirá para identificar aquellos proveedores de los cuales se puedan extraer datos.
- **backend_request:** Siguiendo las directrices vistas en [6.2.1](#), donde se provee un ejemplo para cada caso, es fundamental completar este atributo con la información necesaria para poder obtener información sobre los servicios que ofrece. Veámoslo en detalle:

```
"backend_request": {
  "fetch_method": "<YOUR_FETCH_METHOD>",
  "base_url": ... (If needed)
  "auth": {
    ... (If needed)
  },
  "module": {
```

```

        "func_to_eval": "<MAIN_FUNC>",
        "module_file": "<FILE_NAME>"
    }
}

```

Source Code 4: Formato del atributo backend_request

- **fetch_method**: debe ser “API”, “SDK”, o “WEB-SCRAPING”.
- **base_url**: URL base para las consultas HTTP a la API, o bien para la extracción con *web scraping*.
- **auth**: si fuera necesaria, define la autorización para acceder a los datos. A veces es un *token*, otras veces son variables de entorno.
 - Para los *tokens* o API keys: “Authorization: <PREFIX>TOKEN”, donde *TOKEN* es un literal que será sustituido por la clave presente en la cuenta *dummy*.
 - Para las variables de entorno: lista **env_vars** con los nombres de las variables requeridas. Posteriormente se comprobará su existencia en la cuenta *dummy* designada.
- **module**: indica el nombre del fichero que aloja el código extractor (deberá estar almacenado en la carpeta **modules/source_files/**) y la función a ejecutar para obtener los resultados. Se recomienda el nombre **get_backends** para tal función.
 - Si el método es “API” o “SDK”, esta función principal recibirá por parámetro un objeto de la clase correspondiente (*APIRequest* o *SDKRequest*).
 - Si se trata de una configuración “WEB-SCRAPING”, la función puede asumir que recibirá por parámetro un objeto **webdriver.Chrome** de Selenium para realizar el “raspado” web.

7.2.2. Reconfiguración del sistema

Como se introdujo en esta sección, aunque el sistema es fácilmente extensible, no se ha podido evitar la tarea de realizar algunas modificaciones al código. Aún así, hay que destacar

que estas modificaciones no suponen un cambio en el funcionamiento original del sistema, simplemente añaden características necesarias para terminar de integrar el nuevo proveedor en cuestión. De esta manera, el sistema realmente permanece cerrado a la modificación.

Estos son los pasos a integrar en el código el nuevo proveedor deseado:

1. Si se desea, pueden añadirse datos extra del proveedor que normalmente no serían automatizables. Como una descripción del mismo, o enlaces de interés. Esto puede hacerse incluyendo los datos deseados al fichero `extra_data.py`. El formato se indica dentro del mismo fichero.
2. Es necesario añadir un archivo a la carpeta `modules/source_files/` que contenga el código que extrae los datos del proveedor. El nombre del archivo y la función a ejecutar deberán ser proporcionados en el objeto correspondiente en la base de datos como se indicó en el subapartado 7.2.1 anterior.
3. Es necesario añadir un nuevo modelo en el archivo `backend_models.py` con los atributos que tengan los ordenadores cuánticos obtenidos del nuevo proveedor.

Una vez completados estos pasos, el proveedor quedaría completamente configurado e integrado en el sistema. De este modo, cada vez que el planificador realice una actualización del sistema, los datos de este nuevo proveedor serán recopilados y almacenados junto a los demás.

7.3. Eliminar un Proveedor del Sistema

En caso de que algún proveedor resulte retirado del mercado, o que sus servicios simplemente ya no resulten de interés, es posible eliminarlo del sistema junto con toda su información asociada.

El proceso es tan sencillo como eliminar el proveedor de la base de datos. Al igual que antes, esto puede hacerse por medio de herramientas de MongoDB como *Compass* o *mongosh*, o bien mediante la interfaz definida en las herramientas de administrador (véase la figura 39).

8

Despliegue de la Aplicación

Este capítulo detalla el proceso de despliegue de la aplicación, un paso crucial en el desarrollo del software, pues esto implica poner en producción el sistema desarrollado para que esté disponible al alcance de los usuarios finales.

En este proyecto se ha optado por dos enfoques de despliegue: en la nube y en local. El despliegue en la nube utiliza servicios gratuitos para proporcionar una opción a aquellos que quieran probar la aplicación, sin necesidad de pasar por el proceso de una instalación local. Por otro lado, el despliegue en local tiene como objetivo facilitar la puesta en marcha del sistema en caso de querer tener un control total sobre ella. Este último se realiza por medio de un archivo *Docker Compose* [25].

Enlace para acceder a la plataforma web: <https://quantum-proxy.vercel.app/>²¹.

8.1. Despliegue en la Nube

Como bien expresa su nombre, un despliegue en la nube permite alojar aplicaciones o servicios en servidores remotos, ofreciendo así ventajas como la escalabilidad automática, el mantenimiento simplificado y el acceso global sin necesidad de infraestructura propia. Para este proyecto, se han seleccionado Render, Vercel y MongoDB Atlas como servicios gratuitos para alojar el backend, el frontend y la base de datos, respectivamente.

²¹Debido al uso del plan gratuito de Render, la aplicación puede entrar en modo de suspensión cuando no está en uso, lo que provoca un tiempo de inicio más largo al acceder por primera vez.

8.1.1. Backend con Render

El backend de la aplicación se ha desplegado en Render, una plataforma en la nube que permite alojar aplicaciones web y servicios de backend con facilidad. Render soporta el uso de Docker, lo cual es beneficioso ya que permite empaquetar la aplicación y sus dependencias en un contenedor, asegurando que el entorno de ejecución sea consistente en todos los despliegues. Para desplegar el backend, se ha creado el archivo *Dockerfile* presente en 5, que define los pasos para configurar el entorno y poner en funcionamiento el servicio. Render utilizará este archivo para construir la imagen del contenedor y ejecutarla en su infraestructura.

La elección de Render se basa en su flexible plan gratuito, su facilidad de uso, y su integración continua que permite realizar despliegues automáticos cada vez que se realice un *commit* en la rama principal del proyecto GitHub.

```
# Selecciona Python como imagen base, versión 3.12-slim
FROM python:3.12-slim

# Establece el directorio de trabajo en /code
WORKDIR /code

# Copia el archivo requirements.txt al directorio de trabajo
COPY ./requirements.txt /code/requirements.txt

# Instala las dependencias del archivo requirements.txt
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

# Copia el código del backend a la carpeta app
COPY . /code/app

# Documenta el puerto que expone la aplicación
EXPOSE 80

# Indica el comando que se ejecutará al iniciar el contenedor
CMD ["fastapi", "run", "app/main.py", "--port", "80"]
```

Source Code 5: Dockerfile para el backend

8.1.2. Frontend con Vercel

El frontend, desarrollado con Angular, se ha optado por desplegar en Vercel. A pesar de que Render también ofrece despliegue de archivos estáticos, Vercel es una plataforma de optimizada precisamente para este fin (aplicaciones frontend y sitios estáticos), por lo que al ofrecer también una interfaz intuitiva y despliegue automático ha supuesto una alternativa ideal.

El proceso de despliegue es sencillo, una vez se conecta el repositorio de GitHub con Vercel, este detecta automáticamente el tipo de proyecto (en este caso, Angular) y configura el proceso de construcción y puesta en marcha. Vercel ofrece un dominio predeterminado para la aplicación desplegada, capacidades de redireccionamiento, y la opción de añadir dominios propios. Esto, sin embargo, queda fuera de los requisitos del proyecto, pues con el dominio por defecto es suficiente.

8.1.3. Base de Datos con MongoDB Atlas

Para la base de datos, se ha escogido indudablemente MongoDB Atlas, una plataforma de base de datos como servicio (DBaaS) que proporciona un entorno MongoDB gestionado en la nube. MongoDB Atlas ofrece una capa de servicio gratuita muy generosa, que para proyectos de esta magnitud cumple holgadamente con los requerimientos. La base de datos se puede configurar a través la misma web de MongoDB Atlas, o bien por medio de herramientas como MongoDB Compass, donde se pueden gestionar colecciones, usuarios y permisos de acceso, entre otros.

8.2. Despliegue en Local

Además del despliegue en la nube, se ha configurado un entorno de despliegue local utilizando *Docker Compose*, una herramienta que permite definir y ejecutar aplicaciones con múltiples contenedores de Docker, facilitando el despliegue y la gestión de aplicaciones complejas.

El uso de *Docker Compose* se basa en configurar un archivo `docker-compose.yml`, que posteriormente será manejado con la utilidad de comandos `docker compose`. Entre otros aspectos, tal fichero definirá los servicios que se desplegarán, cómo se construyen, y qué interacciones habrá entre ellos.

La puesta en marcha de la aplicación se describirá en detalle en el apéndice correspondiente al manual de instalación (B), donde también se explicará el significado de las variables de entorno configuradas y cómo puede hacer el usuario para personalizarlas y adaptar el despliegue a sus requerimientos.

No obstante, a continuación se presenta una versión del fichero `docker-compose.yml` simplificada, con el fin de explicar los diferentes servicios que describe y cómo estos interactúan entre sí.

```
services:
  database:
    image: mongo:6.0.13-jammy
    expose:
      - 27017
    volumes:
      - db-data:/data/db
  backend:
    image: fastapi-backend
    build: ./backend
    expose:
      - 8000
    depends_on:
      - database
  frontend:
    image: angular-frontend
    build:
      context: ./frontend
      target: develop
      args:
        - BUILD_MODE=docker
    expose:
      - 80
    depends_on:
      - backend
volumes:
  db-data:
```

Source Code 6: Archivo docker compose simplificado

Como se puede observar en el fragmento de código anterior (6), se definen tres servicios:

- **database:** Se construye usando la imagen oficial de MongoDB para Docker, extraída de DockerHub. Se expone el puerto 27017 y se crea un volumen para almacenar los datos.
- **backend:** Se construye usando el fichero *Dockerfile* correspondiente. Se expone el puerto 8000 y se marca como dependiente del servicio *database*.
- **frontend:** Se construye usando el fichero *Dockerfile* correspondiente. Se expone el puerto 80 y se marca como dependiente del servicio *backend*.

Como se comentaba anteriormente, este enfoque está pensado para proporcionar una alternativa al usuario que quiera tener un control total sobre la aplicación, configurándola acorde a sus necesidades. Además, al estar empaquetada con Docker, permite levantar todo el entorno de la aplicación con un solo comando, simplificando el desarrollo y las pruebas.

Conclusiones y Líneas Futuras

9.1. Conclusiones

9.1.1. Reflexión personal

En lo personal, he querido aprovechar este TFG como un reto personal, donde profundizar más allá de lo visto en la carrera y dándole un enfoque más cercano al desarrollo de software. Durante el desarrollo de este proyecto he podido aplicar conocimientos adquiridos en el grado relativos a la programación, al desarrollo de aplicaciones web, y al proceso de ingeniería de software. Así como también he aprendido nuevas tecnologías, como Python y FastAPI, y nuevos paradigmas, como es el modelo de base de datos no estructurado (o NoSQL) de MongoDB.

A la par, he aprendido mucho sobre Docker, del cuál tenía especial interés. Ha sido utilizado para proporcionar un despliegue local de la aplicación, así como para ejecutar el backend en Render, una plataforma de despliegue que ofrece un plan gratuito.

Desde las fases iniciales, donde la tarea principal fue investigar los distintos proveedores y ponerme en contexto con las tecnologías cuánticas, hasta la implementación y despliegue de un sistema completo con su capa de *backend* y su interfaz *frontend*, considero que en conjunto este trabajo me ha supuesto un desafío del cual quedo orgulloso del resultado, al igual que puedo decir que he aprendido mucho en el camino.

9.1.2. Conclusiones del proyecto

El desarrollo de *Quantum Proxy* ha permitido crear una plataforma centralizada y accesible para la gestión de proveedores de computación cuántica heterogéneos. A lo largo de este proyecto, se han alcanzado varios de los objetivos planteados, tales como la centralización de los

datos de proveedores, la automatización en la actualización de la información y la creación de una interfaz web intuitiva, entre otros. La plataforma facilita el acceso a diferentes máquinas cuánticas y proveedores, mejorando la experiencia de usuario al simplificar la búsqueda por un ordenador cuántico que cumpla las necesidades del usuario.

Uno de los mayores logros del proyecto ha sido la integración exitosa de proveedores como IBM Quantum, IonQ, Rigetti y Amazon Braket, automatizando el sistema en medida de lo posible dada la heterogeneidad inherente entre ellos, así como el uso de módulos que permiten una extensibilidad de la plataforma en caso de añadir más proveedores.

Además, la implementación de servicios como el comparador de proveedores y sistemas cuánticos, junto con el asistente para la selección de máquinas pretende servir de utilidad para incluso aquellos usuarios que no están tan familiarizados con la tecnología cuántica. Y todo esto es posible gracias al sistema de planificación automatizada, que periódicamente refresca los datos almacenados para mantener la plataforma actualizada.

En resumen, *Quantum Proxy* logra el objetivo de ofrecer una solución que facilite la interacción con el ecosistema cuántico actual, proporcionando una herramienta útil tanto para primerizos como para desarrolladores o investigadores.

9.2. Líneas Futuras

Pese a que la plataforma es plenamente funcional, puede ser mejorada en diversos aspectos para expandir su funcionalidad y mejorar aún más la experiencia de usuario. Destacamos los más importantes y que quizás tengan un mayor interés:

1. **Integración de nuevos proveedores:** Por falta de tiempo no se ha añadido *Azure Quantum* como proveedor, pero sería una buena idea incluirlo.
2. **Mejorar el envío de circuitos cuánticos:** Actualmente, únicamente implementa el envío de circuitos al simulador de *IonQ*. Sería muy interesante añadir por ejemplo el envío a *IBM Quantum*, pues constituye una de las plataformas cuánticas más maduras y con mayor soporte en la industria e incluye un plan gratuito.
3. **Migrar el despliegue cloud a otra plataforma:** Si bien en la plataforma se encuentra desplegada usando *Vercel*, *Render* y *MongoDB Atlas*, los planes gratuitos aunque flexibles

tienen sus desventajas. Sería una gran idea migrar los servicios a alguna plataforma como AWS, que asegura un mejor rendimiento, disponibilidad y escalabilidad.

4. **Mejorar el registro a la plataforma:** En este momento la aplicación ofrece su propio servicio de gestión de cuentas de usuario y roles. Sin embargo, puede ser tedioso para el usuario tener que recordar un nuevo inicio de sesión. Sería de utilidad incluir algún tipo de registro e inicio de sesión con *Google* para simplificar esta tarea. Realmente, ya se intentó incluir esto en el proyecto, pero al no ser compatible con el sistema de roles actual, la idea quedó descartada y relevada a una posible línea futura.
5. **Detalles en la experiencia de usuario:** Hay ciertas mejoras que pueden hacerse con respecto a la interfaz web, concretamente sería interesante permitir introducir API keys a partir de un fichero externo, así como para los circuitos, en lugar de tener que meterlos a mano.

Bibliografía

- [1] P. S. Foundation, *Welcome to Python.org*, Version 3.12, Accessed: 2024-08-20. dirección: <https://www.python.org/>.
- [2] S. Ramírez, *FastAPI: Modern, Fast (High-performance), Web Framework for Building APIs with Python*, Accessed: 2024-08-20. dirección: <https://fastapi.tiangolo.com/>.
- [3] S. Project, *Selenium: Browser Automation Tool*, Accessed: 2024-08-20. dirección: <https://www.selenium.dev/>.
- [4] M. Inc., *MongoDB: NoSQL Database*, Accessed: 2024-08-20. dirección: <https://www.mongodb.com/>.
- [5] M. Inc., *MongoDB Compass: GUI for MongoDB*, Accessed: 2024-08-20. dirección: <https://www.mongodb.com/products/compass>.
- [6] G. LLC, *Angular: Web Application Framework*, Version 17.x, Accessed: 2024-08-20, 2024. dirección: <https://angular.dev/>.
- [7] T. B. Team, *Bootstrap: The most popular HTML, CSS, and JS library in the world*, Version 5.3, Accessed: 2024-08-20. dirección: <https://getbootstrap.com/>.
- [8] D. Inc., *Docker: Platform for Developing, Shipping, and Running Applications*, Accessed: 2024-08-20. dirección: <https://www.docker.com/>.
- [9] Render, *Render: Cloud Hosting Platform*, Accessed: 2024-08-20. dirección: <https://render.com/>.
- [10] V. Inc., *Vercel: Frontend Hosting Platform*, Accessed: 2024-08-20. dirección: <https://vercel.com/>.
- [11] M. Inc., *MongoDB Atlas: Multi-Cloud Application Data Platform*, Accessed: 2024-08-20, 2024. dirección: <https://www.mongodb.com/atlas>.
- [12] M. Corporation, *Visual Studio Code: Code Editor Redefined*, Accessed: 2024-08-20. dirección: <https://code.visualstudio.com/>.
- [13] A. C. Plc, *Trello: Visual Collaboration Tool*, Accessed: 2024-08-20, 2024. dirección: <https://trello.com/>.

- [14] I. GitHub, *GitHub: Where the World Builds Software*, Accessed: 2024-08-20. dirección: <https://github.com/>.
- [15] I. Amazon Web Services, *¿Qué es la computación cuántica?* Accessed: 2024-08-24, 2024. dirección: <https://aws.amazon.com/es/what-is/quantum-computing/>.
- [16] P. y Coma, *Google Vuelve a la Carga: Ahora la Computación Cuántica*, Accessed: 2024-08-26, 2023. dirección: <https://medium.com/punto-y-coma/google-vuelve-a-la-carga-ahora-la-computaci%C3%B3n-cu%C3%A1ntica-3c9897d72bdb>.
- [17] J. Alvarado-Valiente, J. Romero-Álvarez, E. Moguel, J. García-Alonso y J. M. Murillo, «Technological diversity of quantum computing providers: a comparative study and a proposal for API Gateway integration,» *Software Quality Journal*, vol. 32, n.º 1, págs. 53-73, mar. de 2024.
- [18] I. Amazon Web Services, *Amazon Braket: Quantum Computing as a Service*. dirección: <https://aws.amazon.com/braket/>.
- [19] I. Corporation, *IBM Quantum: Explore Quantum Computing in the Cloud*. dirección: <https://www.ibm.com/quantum-computing/>.
- [20] I. IonQ, *IonQ: Trapped-Ion Quantum Computing*. dirección: <https://ionq.com/>.
- [21] R. Computing, *Rigetti: Quantum Computing Platform*. dirección: <https://www.rigetti.com/>.
- [22] PennyLaneAI, *PennyLane: A Python Library for Quantum Machine Learning, Automatic Differentiation, and Optimization of Hybrid Quantum-Classical Computations*, Accessed: 2024-08-25, 2024. dirección: <https://github.com/PennyLaneAI/pennylane?tab=readme-ov-file#key-features>.
- [23] E. R. Johnston, N. Harrigan y M. Gimeno-Segovia, *Programming Quantum Computers: Essential Algorithms and Code Samples*. Sebastopol, CA, USA: O'Reilly Media, 2019, ISBN: 978-1492039686.
- [24] M. Corporation, *Azure Quantum: Quantum Computing in the Cloud*, Accessed: 2024-08-22, 2024. dirección: <https://azure.microsoft.com/en-us/products/quantum/>.
- [25] D. Inc., *Docker Compose: Define and Run Multi-Container Applications with Docker*, Accessed: 2024-08-20. dirección: <https://docs.docker.com/compose/>.

Apéndice A

Manual de Uso

Este apéndice presenta un manual de uso de la plataforma, detallando todas las funcionalidades y características disponibles. Se recuerda al lector que puede accederse a la aplicación por dos medios:

- Mediante internet a través del enlace <http://quantum-proxy.vercel.app>.
- Mediante una instalación local siguiendo los pasos del apéndice B.

En este manual se detallan instrucciones para navegar por la interfaz e interactuar con la misma, así como el uso del *Wizard* y el envío de circuitos para su ejecución en los proveedores.

En el apartado A.8 se ofrece una cuenta probar la aplicación al completo.

A.1. Página principal

Al acceder por primera vez a la plataforma el usuario se encontrará con la página principal (figura 25), desde donde es posible navegar a cualquiera de los servicios ofrecidos mediante los enlaces existentes o haciendo uso de la barra de navegación principal.

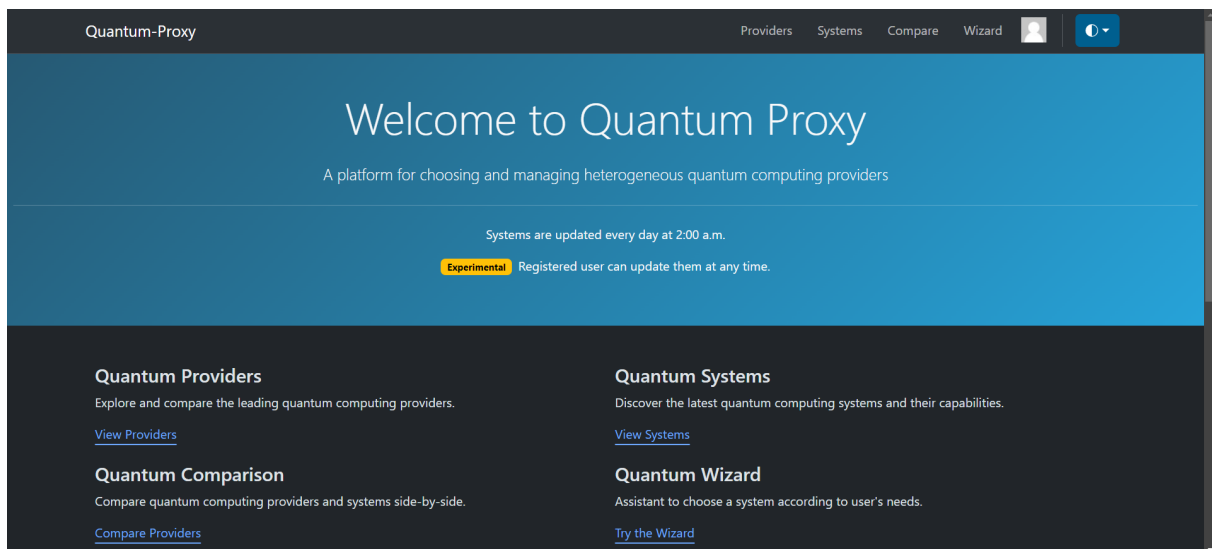


Figura 25: Fragmento de la página principal

A.2. Registro

Para poder acceder a todos los servicios al completo, es recomendable registrarse en la plataforma. Registrarse hace posible el acceso dos funcionalidades fundamentales:

- Posibilita el envío de trabajos a proveedores cuánticos, y el acceso a sus resultados.
- De manera experimental, ofrece la posibilidad de refrescar la información de los proveedores y sus máquinas cuánticas desde la vista de *Systems* A.7.2.

El proceso para registrarse es el siguiente:

- Pulsar sobre el icono de usuario no autenticado (junto al selector de temas), se abrirá un desplegable con la única opción “Login or Sign-up”.
- Tras pulsar en la opción del desplegable, el usuario será redirigido a la página de inicio de sesión.
- Una vez aquí, pulsar sobre el botón “Register”, que redirigirá una vez más al usuario a la página de registro apreciada en 26.

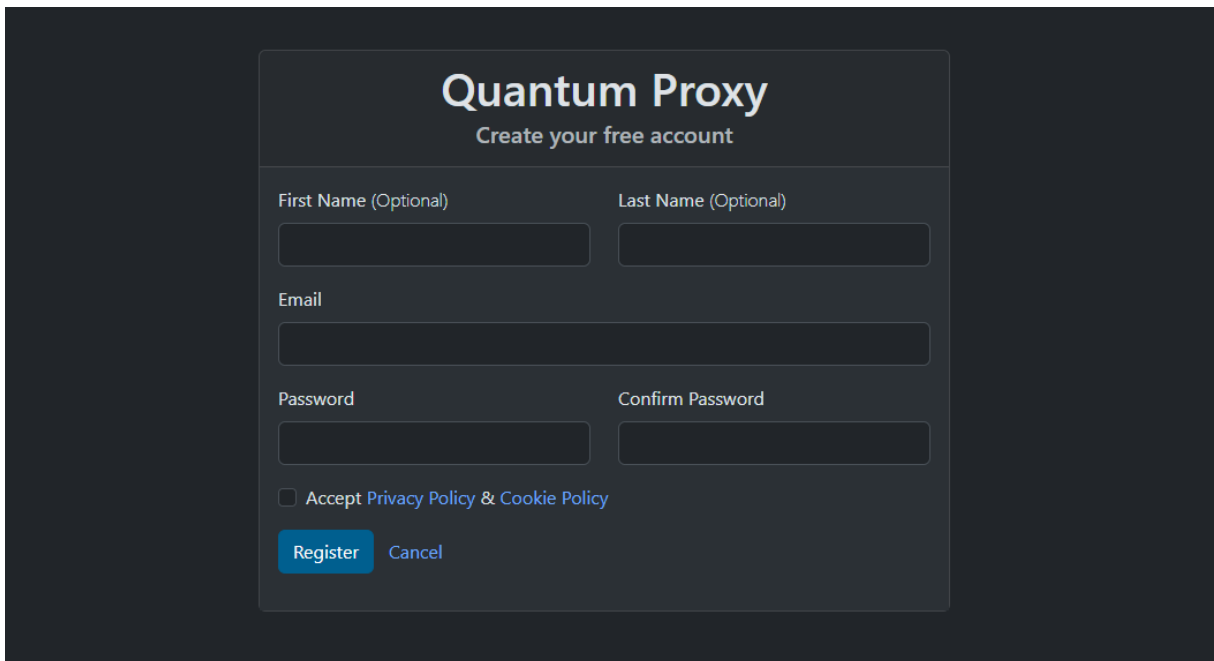


Figura 26: Página de registro

Como se puede observar en la figura anterior, los únicos campos obligatorios a rellenar son el email y la contraseña, siendo el nombre y apellido opcional.

Una vez completado el formulario con los datos, y aceptada la política de privacidad y *cookies*²², al pulsar en “Register” se pondrá en marcha el proceso de registro, el cual eventualmente enviará un correo de verificación al usuario para activar así su cuenta.

A.3. Inicio de sesión

El proceso de inicio de sesión es similar al de registro. El usuario deberá pulsar en el icono de usuario no autenticado (junto al selector de temas) para acceder a la vista de inicio de sesión. Una vez allí, deberá ingresar su email y contraseña para iniciar sesión en la aplicación.

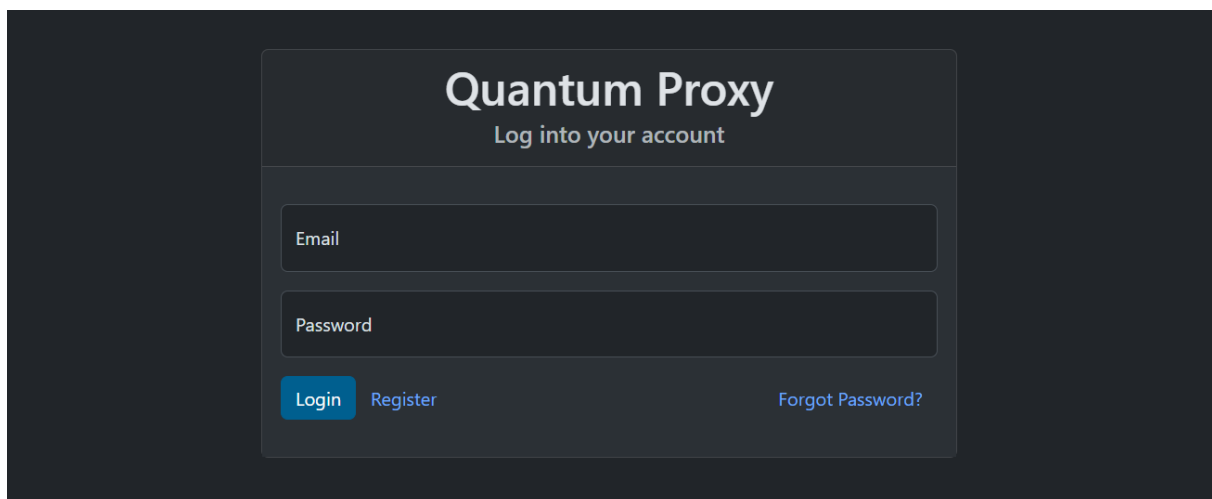


Figura 27: Página de inicio de sesión

La aplicación hace uso de tokens JWT para autenticar al usuario, así como de *refresh tokens* que permiten que el usuario no tenga que autenticarse nuevamente cada vez que el token JWT expire. De este modo, se garantiza simplicidad y facilidad en la experiencia del usuario.

²²Aunque se hable de política de cookies, la aplicación únicamente hace uso de una cookie funcional para obtener un avatar en función del email del usuario. No obstante, se ha considerado de interés informar al usuario de ello, así como proporcionar una política de privacidad que garantiza el correcto tratamiento de sus datos personales.

A.4. Recuperación de contraseña

Junto con el servicio de login y de registro se incluye una funcionalidad de recuperación de contraseña. En caso de haber olvidado la contraseña con la que se accede a la plataforma, el usuario puede solicitar una nueva contraseña mediante el formulario 28, accesible desde la vista de inicio de sesión (véase la figura 27).

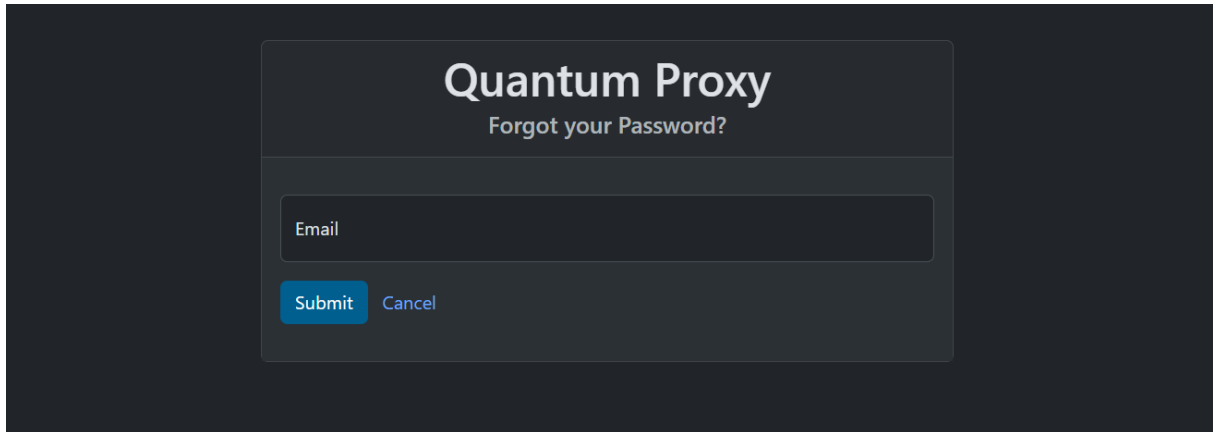
The image shows a dark-themed web interface for 'Quantum Proxy'. At the top, the text 'Quantum Proxy' is displayed in a large, white, sans-serif font, with 'Forgot your Password?' in a smaller font below it. Below the text is a form with a single input field labeled 'Email'. At the bottom of the form are two buttons: a blue 'Submit' button and a light blue 'Cancel' button.

Figura 28: Página de recuperación de contraseña

El proceso es sencillo, una vez el usuario introduce sus datos y los envía, le llegará un correo al email designado con un enlace que le redirigirá a la ventana 29. Una vez aquí, puede definir una nueva contraseña para su cuenta en la plataforma.

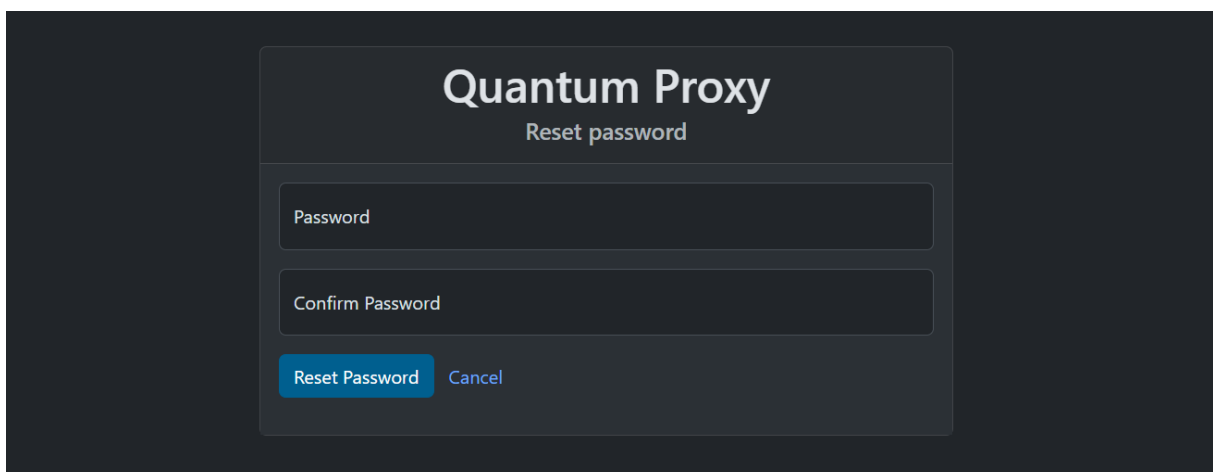
The image shows a dark-themed web interface for 'Quantum Proxy'. At the top, the text 'Quantum Proxy' is displayed in a large, white, sans-serif font, with 'Reset password' in a smaller font below it. Below the text is a form with two input fields: 'Password' and 'Confirm Password'. At the bottom of the form are two buttons: a blue 'Reset Password' button and a light blue 'Cancel' button.

Figura 29: Página de cambio de contraseña

A.5. Menú desplegable para usuarios autenticados

Si el usuario ya dispone de una cuenta y ha iniciado sesión en la plataforma, el icono de usuario no autenticado pasará a mostrar un avatar único basado en el hash del email registrado (vía Gravatar ²³).

Al clicar en él, aparecerá un desplegable con muchas más opciones, similar al de la figura 30 mostrada a continuación.

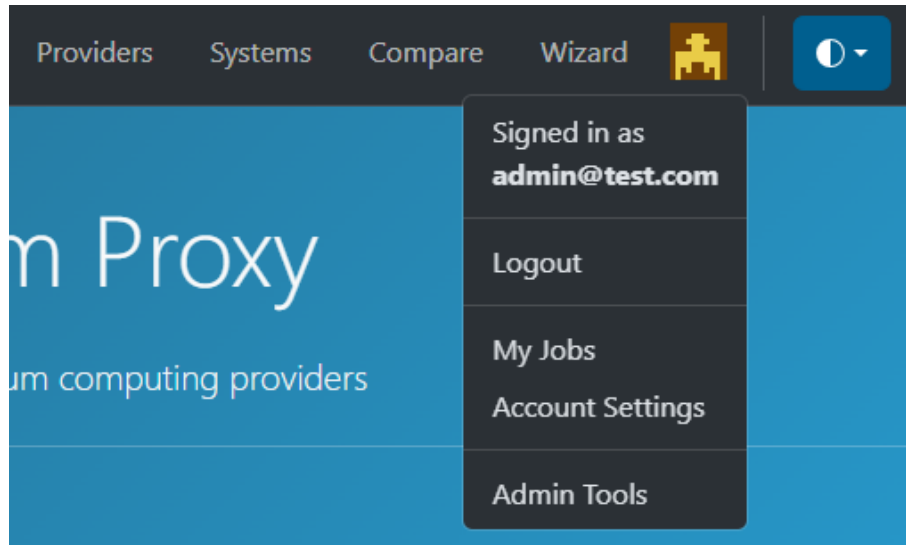


Figura 30: Menú desplegable para usuarios autenticados

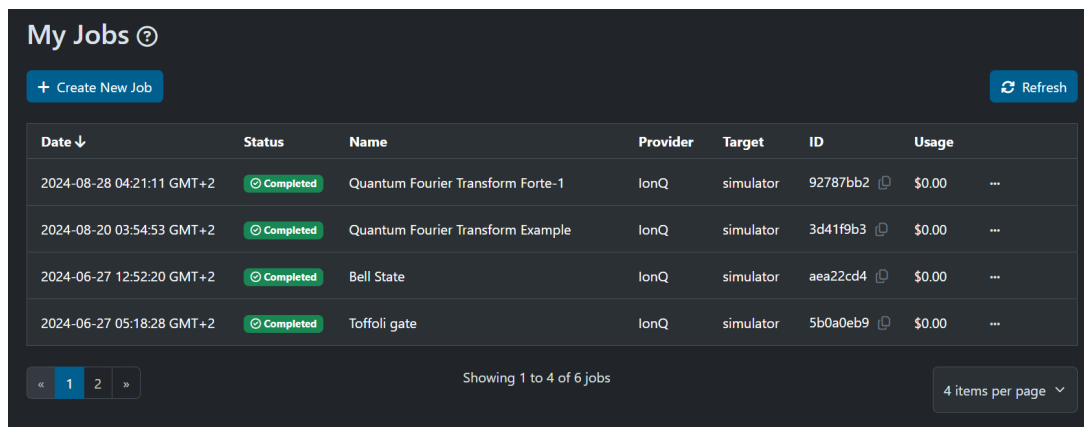
- **Logout:** permite cerrar sesión, retornando a la página principal.
- **My Jobs:** permite acceder al panel de trabajos, donde se podrán enviar circuitos a proveedores de computación cuántica y visualizar sus resultados (apdo. A.5.1 del manual).
- **Account Settings:** información varia del usuario con opción a actualizarla en cualquier momento. Incluye datos personales y diccionario de API keys asociado al usuario.
- **Admin Tools** (solo para usuarios con el rol de administrador): permite acceder a una vista exclusiva para administradores. En ella se pueden manejar cuentas de usuario, así como realizar actualizaciones manuales sobre los datos obtenidos de los proveedores.

Tales opciones serán detalladas en los subapartados a continuación.

²³<https://gravatar.com/>

A.5.1. My Jobs

La vista “My Jobs” (figura 31) ofrece un panel de trabajos asociado a cada usuario. Basándose en sus API keys, recopila todos los trabajos enviados y obtiene sus resultados para mostrarlos de manera ordenada e interactiva.

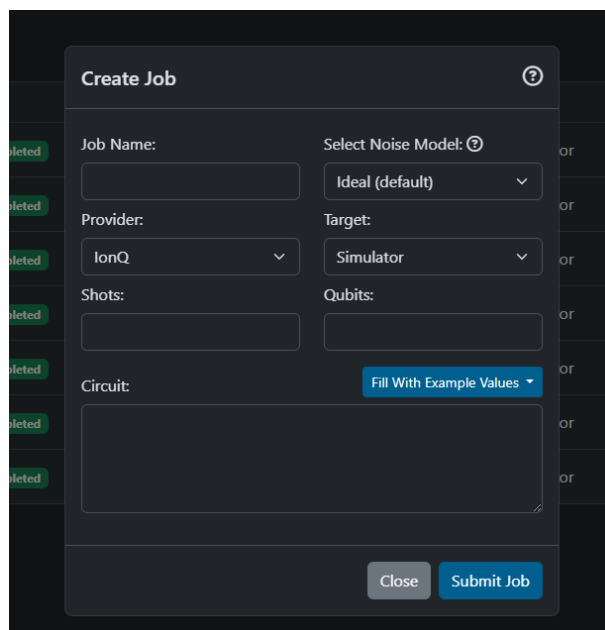


The screenshot shows the 'My Jobs' interface with a table of completed jobs. The table has columns for Date, Status, Name, Provider, Target, ID, and Usage. There are four jobs listed, all with a 'Completed' status and a usage of '\$0.00'. Below the table, there is a pagination control showing 'Showing 1 to 4 of 6 jobs' and a '4 items per page' dropdown.

Date ↓	Status	Name	Provider	Target	ID	Usage
2024-08-28 04:21:11 GMT+2	Completed	Quantum Fourier Transform Forte-1	IonQ	simulator	92787bb2	\$0.00
2024-08-20 03:54:53 GMT+2	Completed	Quantum Fourier Transform Example	IonQ	simulator	3d41f9b3	\$0.00
2024-06-27 12:52:20 GMT+2	Completed	Bell State	IonQ	simulator	aea22cd4	\$0.00
2024-06-27 05:18:28 GMT+2	Completed	Toffoli gate	IonQ	simulator	5b0a0eb9	\$0.00

Figura 31: Vista de trabajos del usuario

Si se pulsa el botón “Create New Job”, aparece en pantalla una ventana modal que permite configurar un trabajo a ejecutar. El contenido de la ventana es el representado en la siguiente imagen 32.



The screenshot shows the 'Create Job' modal form. It contains fields for Job Name, Provider (set to IonQ), Shots, and Circuit. There are also dropdowns for Select Noise Model (set to Ideal (default)) and Target (set to Simulator). A 'Fill With Example Values' button is next to the Circuit field. At the bottom, there are 'Close' and 'Submit Job' buttons.

Figura 32: Modal para crear un trabajo

En dicha ventana encontramos un formulario para generar y enviar²⁴ el trabajo deseado, contando con campos como:

- **Job Name:** nombre del trabajo.
- **Noise Model:** al disponer de envíos de trabajos al simulador de IonQ, se brinda la posibilidad de elegir un “modelo de ruido” para añadir a la simulación. Este ruido simula un entorno en hardware cuántico real.
- **Provider:** elección del proveedor de computación cuántica.
- **Target:** elección de la máquina cuántica donde se ejecutará el circuito.
- **Shots:** indica cuántas veces se ejecutará el circuito.
- **Qubits:** indica el número de *qubits* del circuito.
- **Circuit:** define el circuito siguiendo el formato definido por IonQ²⁵

Entre los elementos del formulario se puede apreciar un botón “Fill With Example Values”, el cual al presionarse abre un desplegable con distintos ejemplos de circuitos cuánticos. Se anima al usuario a probar cada uno de ellos, así como modificar el modelo de ruido para ver cómo altera los resultados.

Volviendo a la vista general de trabajos, el panel de la imagen 31 ofrece una vistazo general y rápido a los trabajos enviados por el usuario, no obstante, si se requiere conocer los resultados de alguno de ellos, basta con clicar la fila de la tabla correspondiente al trabajo deseado.

Esta acción redirigirá al usuario a una nueva vista similar a la de la imagen 33, que representa un trabajo que ejecutó 1000 veces la Transformada Cuántica de Fourier con el modelo de ruido de Aria-2.

²⁴Para poder enviar trabajos es requerido contar con las API keys correspondientes. Siguiendo el formato de la figura 35, para enviar tareas a IonQ se debe definir ‘ionq’ como plataforma y ‘TOKEN’ como *key type*.

²⁵Consulte [Writing a quantum circuit](#) y [Supported Gates](#), por IonQ Docs.

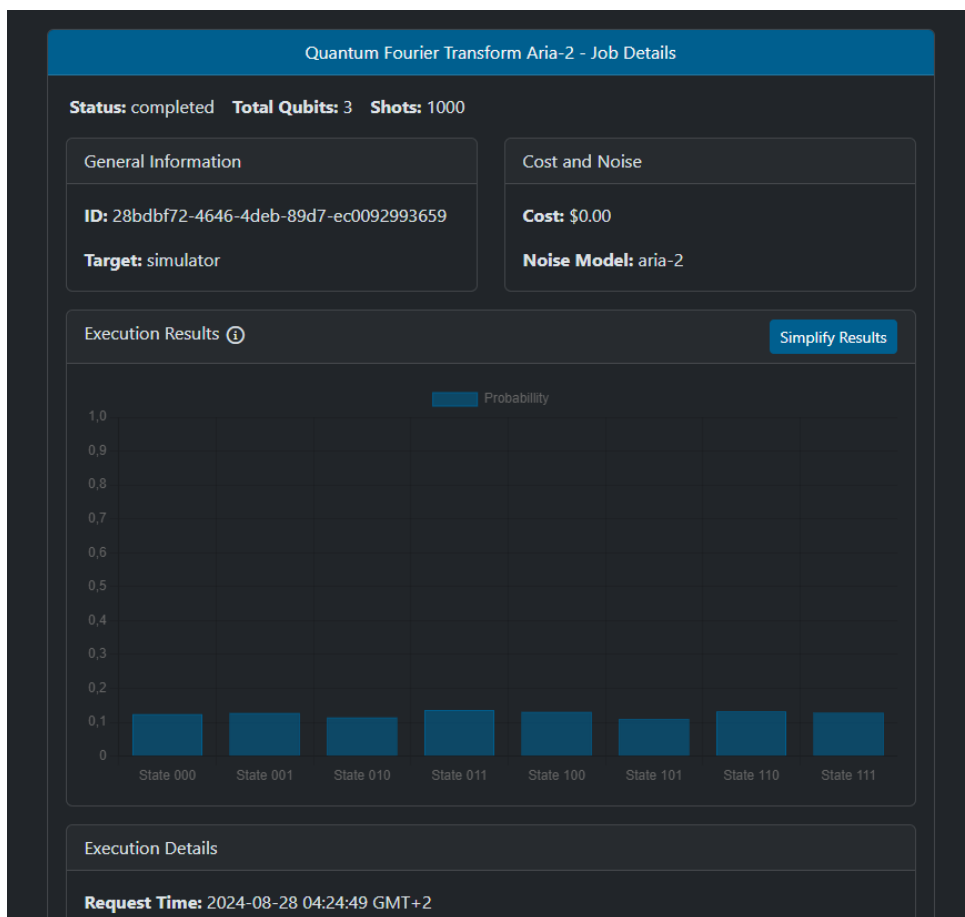


Figura 33: Fragmento de los detalles de un trabajo

El botón “Simplify Results” que aparece sobre la gráfica alterna entre dos modos de visualización. En el modo por defecto muestra todos los 2^n estados, donde n es el número de *qubits* utilizados en el circuito. En el segundo modo, muestra únicamente aquellos estados cuya probabilidad es mayor que cero.

A.5.2. Account Settings

Mediante esta vista²⁶ el usuario puede acceder a la información almacenada sobre él en la plataforma (datos personales y API keys) y actualizarla cuando estime oportuno. Con un clic en el botón de “Update Profile”, pasamos a la vista de la figura 34 para editar los datos del usuario.

²⁶Dada la simplicidad de la vista, se ha considerado no incluir una captura de esta.

Update Profile

First Name: admin

Last Name: admin

Email: admin@uma.es

API Keys & Secrets: [Update]

Change Password

Leave blank to keep the same password

Password: []

Confirm Password: []

[Update] [Delete] [Cancel]

Figura 34: Vista para editar datos del usuario

Dejando de lado atributos como el nombre o el email, quizás lo más destacable sea el apartado de “API Keys & Secrets”, donde al pinchar en él aparecerá una ventana modal (figura 35) para visualizar y editar las API keys de manera sencilla e intuitiva.

Update Secrets [X]

Platform	Key Type	Value		
ionq	TOKEN	n2QwQpeJTvllTs1oR/	-	+
ibm	TOKEN	4e93d7704a95e7580	-	+
			-	+

[Close] [Save Changes]

Figura 35: Modal para editar las API keys y secretos del usuario

A.5.3. Admin Tools

Como se mencionó anteriormente, esta vista es exclusiva para usuarios con el rol de administrador, el cual únicamente puede ser concedido por otro administrador.

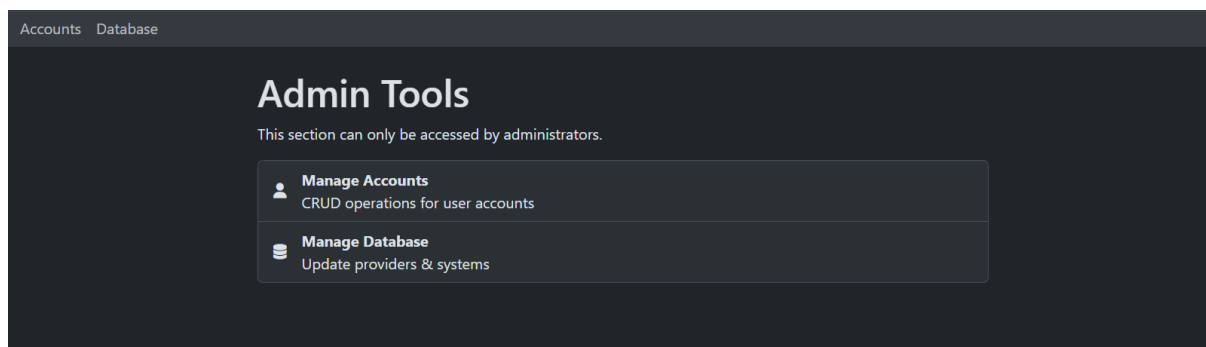


Figura 36: Modal para editar las API keys y secretos del usuario

Dentro de este área, se ofrecen dos herramientas principales:

- **Manage Accounts:** herramientas para la gestión de usuarios.
- **Manage Database:** herramientas para la gestión de la base de datos.

Al pulsar en cualquiera de ellas, o bien en la nueva barra de navegación secundaria que aparece en la parte superior, redirigirá al usuario a la respectiva página de administración.

Para el primer caso de gestión de cuentas, se tiene la vista representada en la imagen a continuación (37). Donde se permite crear nuevas cuentas, editar las existentes, o eliminarlas.

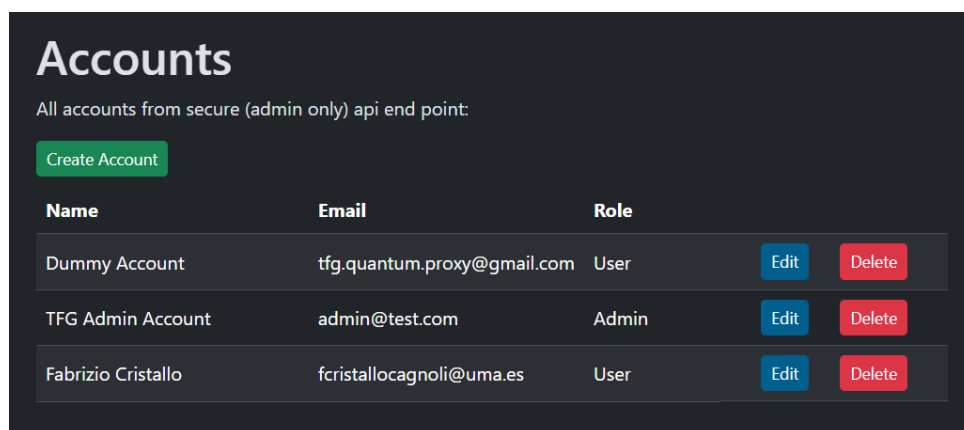


Figura 37: Lista de cuentas del sistema

Para el segundo caso, gestión de la base de datos, se tiene la vista referida en la imagen 38. Donde se permite actualizar la información sobre los sistemas, añadir nuevos proveedores, o modificar los existentes.

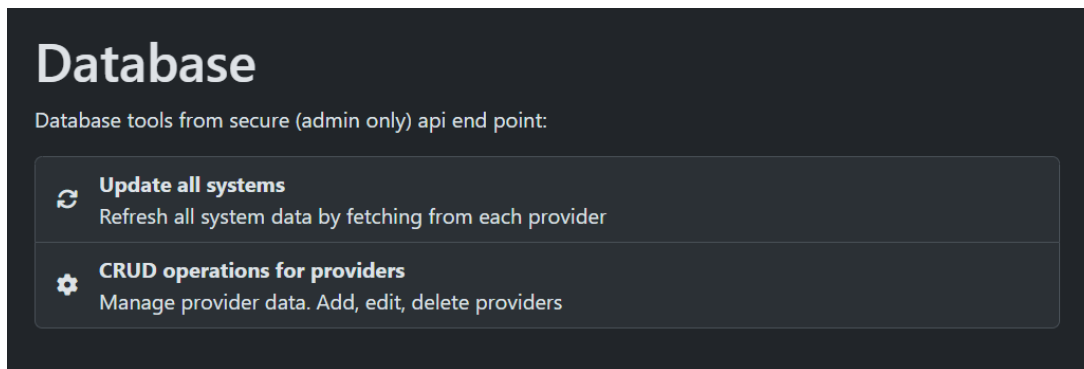


Figura 38: Herramientas para la gestión de la base de datos

- El botón “Update all systems” actualizará la información de todos los sistemas almacenados.
- Pulsar el botón “CRUD operations for providers” ofrece una nueva vista (figura 39) para administrar los proveedores existentes en la base de datos. Permitiendo añadir nuevos, modificar sus parámetros, o eliminarlos.

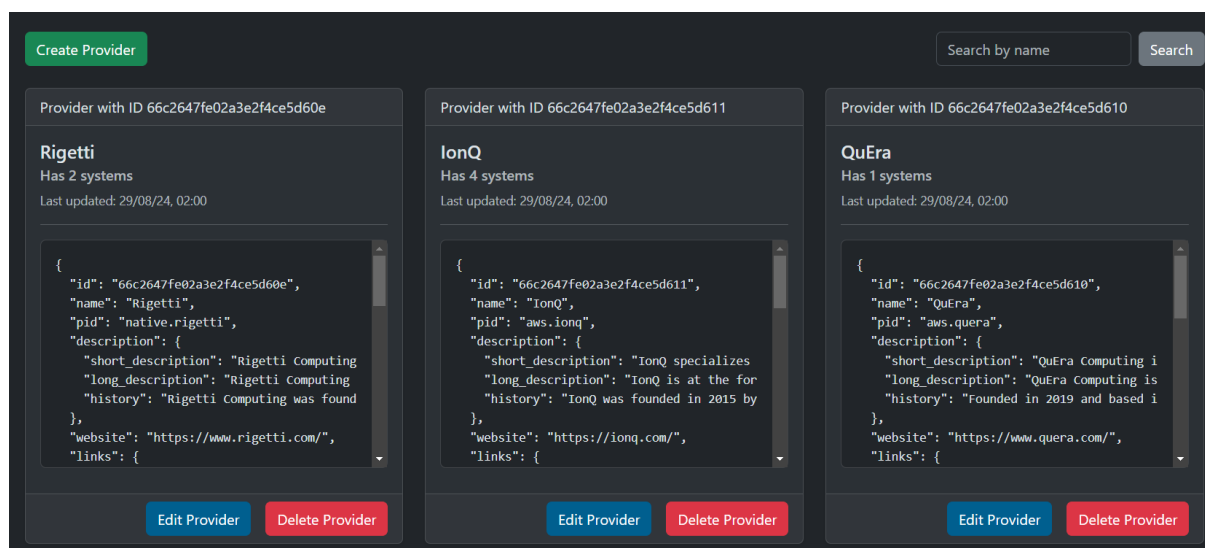


Figura 39: Vista para administrar proveedores

Aunque se ofrezca esta vista para gestionar proveedores, es muy recomendable el uso de la herramienta *MongoDB Compass*, ya que esta proporciona una interfaz dedicada para el manejo de bases de datos MongoDB.

A.6. Selección de tema de color

En la esquina superior derecha se encuentra el botón de selección de tema de color (figura 40). Como en muchas aplicaciones, se ofrece un tema claro y otro oscuro para que el usuario escoja con el que se sienta más cómodo.

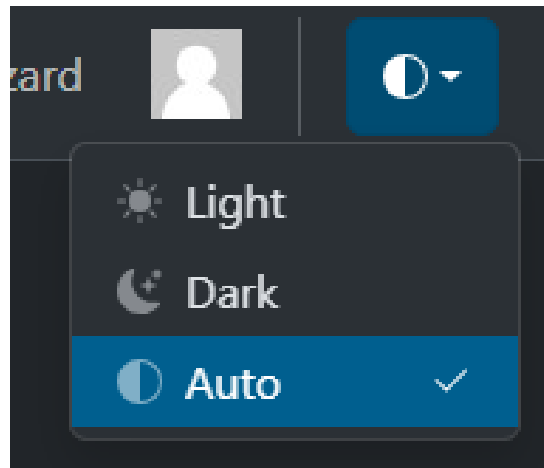
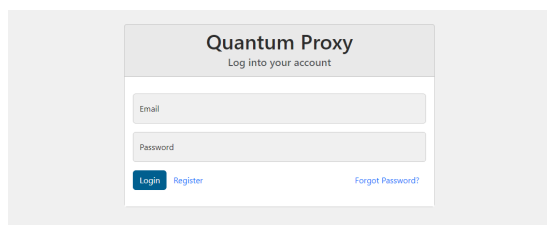
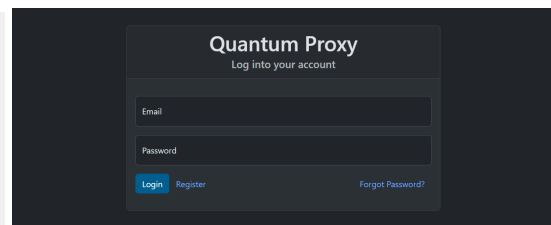


Figura 40: Selector de temas

El desplegable ofrece tres opciones: *Light*, *Dark* y *Auto*²⁷; siendo esta última opción la seleccionada por defecto. Podemos ver una demostración de dichos temas en las figuras 41a y 41b a continuación.



(a) Tema claro



(b) Tema oscuro

Figura 41: Ejemplo de visualización de ambos temas

²⁷El modo *Auto* consulta la opción preferida por el usuario mediante el atributo *prefers-color-scheme*.

A.7. Barra de navegación

Vistos el menú desplegable de usuario autenticado, y el selector de temas, seguidamente se detallará el resto de elementos de la barra de navegación superior, el cual permite navegar por todas las vistas principales de la plataforma.

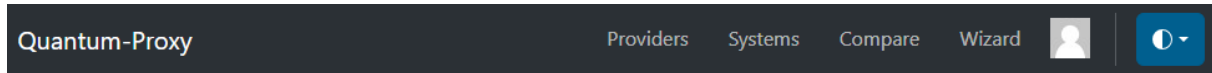


Figura 42: Barra de navegación principal

A.7.1. Providers

Providers es la vista que ilustra todos los proveedores recogidos en la base de datos. En ella se encuentra información como: el método mediante el cual son recogidos sus datos, su última fecha de actualización, y cuántos sistemas ofrecen.

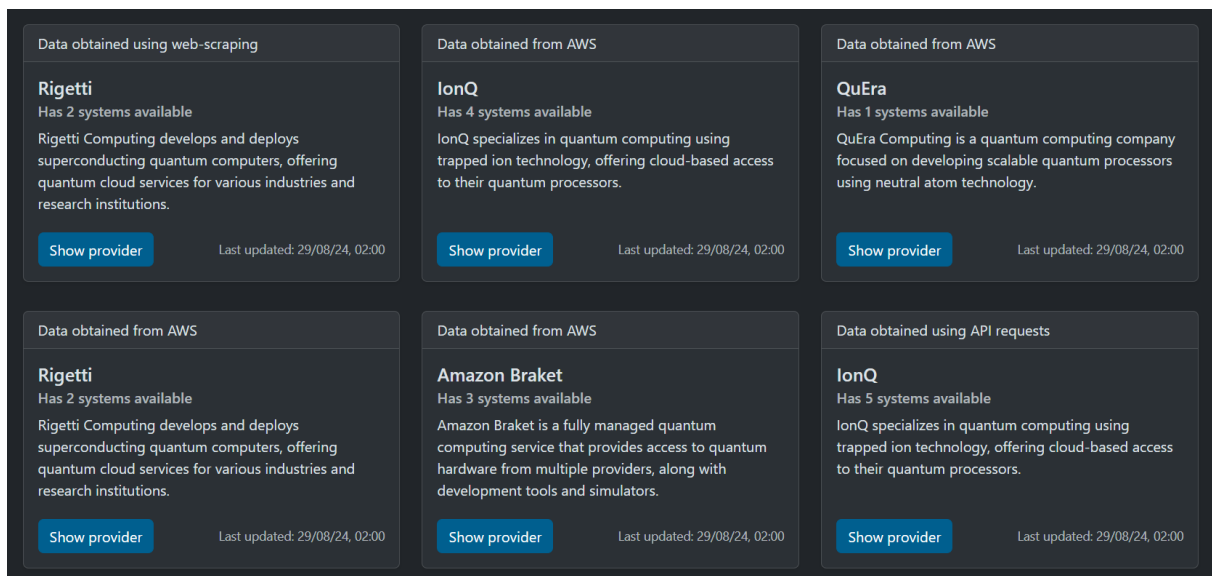


Figura 43: Vista de proveedores

Clicando en el botón “Show provider” el usuario será redirigido a la vista específica de un proveedor (ejemplo en figura 44). Esta vista ofrece elementos como: descripciones del proveedor, página web del mismo y sistemas disponibles (y cuántos de ellos están *online* o disponibles).

IonQ

Summary

IonQ specializes in quantum computing using trapped ion technology, offering cloud-based access to their quantum processors.

Description

IonQ is at the forefront of quantum computing with its unique trapped ion technology. The company's quantum computers leverage individual ions, which are electrically charged atoms, to function as qubits. This approach offers high levels of stability and coherence, leading to longer qubit operation times and more accurate computations. IonQ provides access to its quantum systems through major cloud platforms like Amazon Web Services (AWS), Microsoft Azure, and Google Cloud, enabling researchers, developers, and businesses to experiment and develop applications in quantum computing.

History

IonQ was founded in 2015 by Chris Monroe and Jungsang Kim, both pioneers in the field of quantum computing. The company spun out of research conducted at the University of Maryland and Duke University. IonQ quickly established itself as a leader in the quantum space, securing significant funding and partnerships with tech giants. Their technology, based on decades of academic research, continues to push the boundaries of what's possible with quantum computing.

[View website](#)

Has 4 systems available,
of which 2 are online:

Aria 1 - from IonQ (by AWS)

Last updated: 29/08/24, 02:00

Qubits	Queued Jobs
25	7
Price	Status
\$0.3/task	Online

Show system

[View all systems](#)

A.7.2. Systems

From IBM Quantum Kyiv 127 qubits Status: 🟢 Available Queued Jobs: 994 Show System Last updated: 29/08/24, 02:00	From IBM Quantum Sherbrooke 127 qubits Status: 🟢 Available Queued Jobs: 1984 Show System Last updated: 29/08/24, 02:00	From IBM Quantum Kyoto 127 qubits Status: 🟢 Available Queued Jobs: 94 Show System Last updated: 29/08/24, 02:00	From IBM Quantum Brisbane 127 qubits Status: 🟢 Available Queued Jobs: 1944 Show System Last updated: 29/08/24, 02:00
From Amazon Braket (by AWS) dm1 17 qubits Status: 🟢 Online Queued Jobs: 0 Show System Last updated: 29/08/24, 02:00	From Amazon Braket (by AWS) TN1 50 qubits Status: 🟢 Online Queued Jobs: 0 Show System Last updated: 29/08/24, 02:00	From Amazon Braket (by AWS) SV1 34 qubits Status: 🟢 Online Queued Jobs: 0 Show System Last updated: 29/08/24, 02:00	From IonQ (by AWS) Harmony 11 qubits Status: 🔴 Offline Queued Jobs: 0 Show System Last updated: 29/08/24, 02:00

En el encabezado de cada tarjeta se especifica la procedencia de cada máquina, indicando el proveedor y la fuente de terceros si corresponde (ej. AWS).

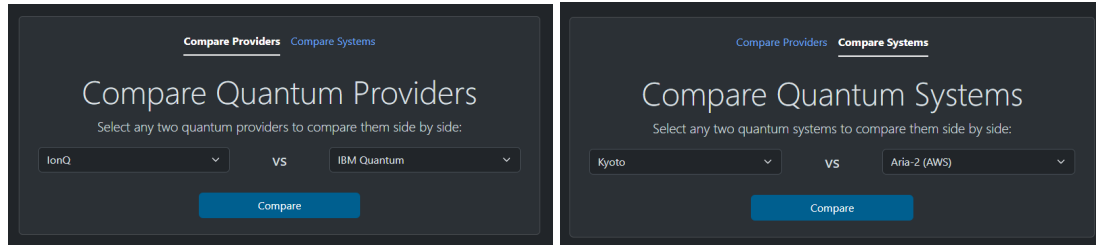
Al igual que en la página de los proveedores, existe un botón “Show System” que permite ver la información al completo de un sistema. En la figura 46 se presenta un ejemplo de dicha vista.

Kyoto — IBM Quantum	
Common Fields	
Backend Name	Kyoto
Provider	IBM Quantum
Status	🟢 Available
Qubits	127
Queued Jobs	94
Extra Fields	
Basis Gates	ecr, id, rz, sx, x
Clops H	5000
Credits Required	true
Max Experiments	300
Max Shots	100000
Last Checked	29/08/24, 02:00

Figura 46: Vista específica del sistema Kyoto, por IBM Quantum

A.7.3. Compare

Compare es la vista que ofrece el servicio de comparativa. Esta funcionalidad permite comparar dos proveedores o dos máquinas cuánticas.



(a) Comparativa de proveedores

(b) Comparativa de sistemas

Figura 47: Servicio de comparativa

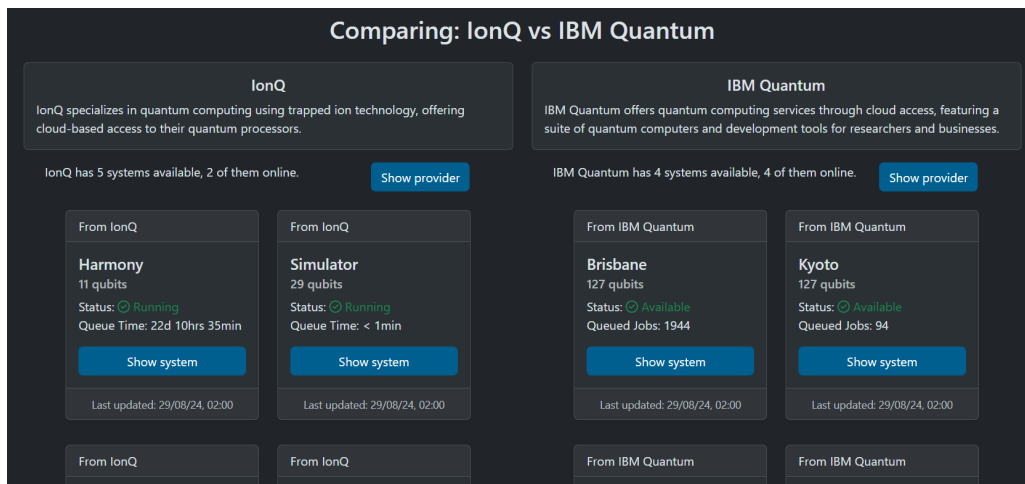


Figura 48: Ejemplo de comparativa de proveedores

A.7.4. Wizard

Wizard es el servicio de asistente para la elección de una máquina cuántica. Esta vista ofrece un vistazo general a todos los sistemas agrupados por proveedores, así como la posibilidad de filtrar los resultado en función de las necesidades del usuario. Las figuras 49 y 50 muestran el contenido de la página en cuestión.

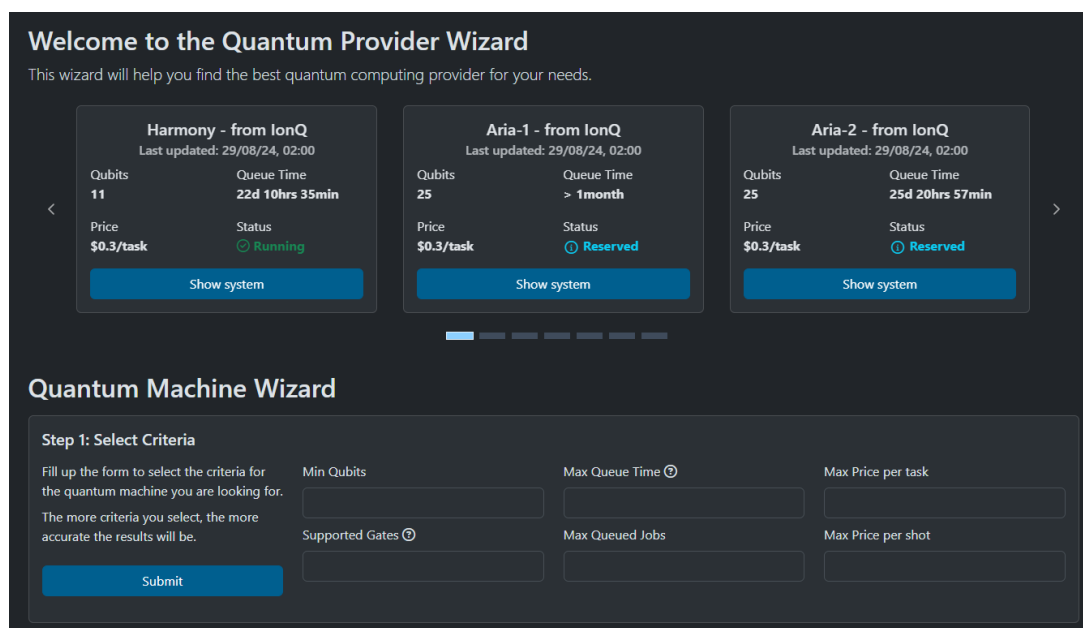


Figura 49: Fragmento 1 de la página del Wizard

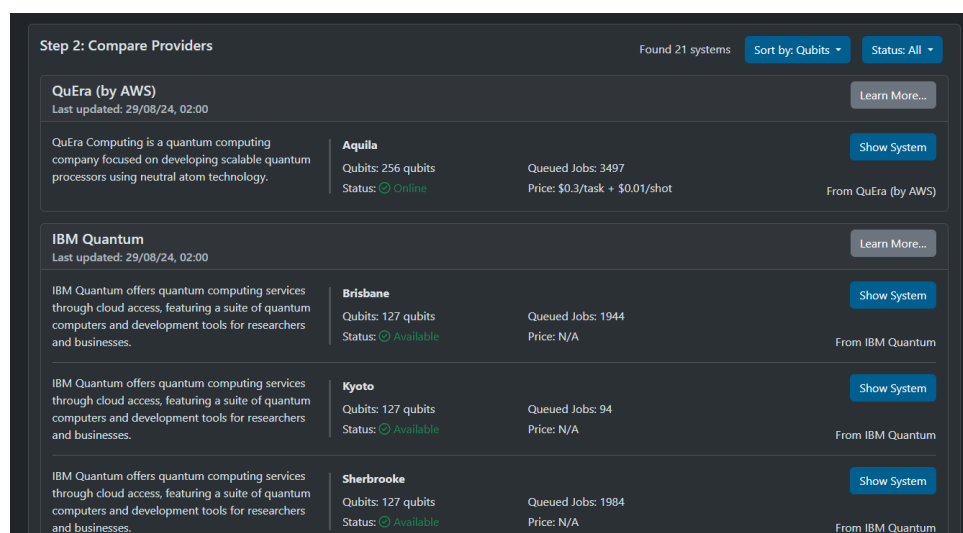


Figura 50: Fragmento 2 de la página del Wizard

A.8. Acceso privilegiado a la plataforma

Para finalizar, cabe destacar que se ofrece una cuenta con privilegios de administrador para tener acceso y probar todas las funcionalidades de la aplicación, así como la posibilidad de crear nuevas cuentas. Una vez finalizada la defensa del TFG, dicha cuenta perderá su rol de administrador y pasará a comportarse como un usuario más.

Las claves de acceso son las siguientes:

- **Usuario:** admin@test.com
- **Contraseña:** admintesting

El motivo por el cual se deshabilitarán los privilegios de administrador es sencillo. Puesto que son claves para entrar a una aplicación desplegada en internet, donde todo el mundo tiene acceso, cualquiera con acceso a esta memoria (y por siguiente a las claves) podría perjudicar al resto de usuarios registrados y acceder a información confidencial al tener acceso a todas las cuentas desde el panel de administrador.

Apéndice B

Manual de Instalación

Este apéndice proporciona un manual o guía de instalación de la aplicación mediante el uso de la herramienta *Docker Compose*. Permitirá desplegar la plataforma en un entorno local, totalmente configurable para las necesidades del usuario.

B.1. Requisitos previos a la instalación

Para comenzar con la instalación no se requiere de especial material, bastará con un ordenador con acceso a internet y que tenga *Docker*²⁸ y *Docker Compose*²⁹ previamente instalados. No es estrictamente necesario para el proyecto, pero es recomendable tener instalado también la herramienta *Git*³⁰.

B.2. Descarga del código fuente

El primer paso será descargar el código fuente de la aplicación, el cual se encuentra alojado en el repositorio <https://github.com/fcristallocagnoli/Quantum-Proxy>, en GitHub. Este puede obtenerse bien clonando el repositorio con el comando `git clone` o bien descargado el proyecto como archivo ZIP y descomprimiéndolo. Una vez hecho esto, entramos en la carpeta del proyecto `Quantum-Proxy/`.

B.3. Configuración del archivo `docker-compose.yml`

Antes de entrar en detalle de los pasos para levantar los servicios que conforman la aplicación, es necesario comprender la configuración existente en el fichero `docker-compose.yml`. De esta forma, el usuario podrá adaptar el funcionamiento de la aplicación a sus necesidades.

²⁸<https://docs.docker.com/engine/install/>

²⁹<https://docs.docker.com/compose/install/>

³⁰<https://git-scm.com/downloads>

A continuación (figura 7) se presenta el contenido de tal archivo de configuración. Iremos detallando servicio a servicio el significado de las variables de entorno y cómo personalizarlos.

```
services:
  database:
    image: mongo:6.0.13-jammy
    restart: always
    environment:
      DB_DATABASE: quantum-proxy-db
      DB_ADMIN_USER: ${DB_ADMIN_USER:-admin}
      DB_ADMIN_PASSWORD: ${DB_ADMIN_PASSWORD:-admin}
      DB_DUMMY_ACCOUNT: tfg.quantum.proxy@gmail.com
    ports:
      - "27017:27017"
    expose:
      - 27017
    volumes:
      - db-data:/data/db
      - ./backend/database/mongo-init.js:/docker-entrypoint-initdb.d/mongo-init.js:ro
  backend:
    build: ./backend
    environment:
      FRONTEND_URL: "http://frontend"

      DB_URI: "mongodb://admin:admin@database"
      DB_PORT: 27017

      DUMMY_ACCOUNT: tfg.quantum.proxy@gmail.com

      EMAIL_SENDER: tfg.quantum.proxy@gmail.com
      EMAIL_APP_PASSWORD: "tfbr yanj aiwj mvnv"
      EMAIL_ACCOUNT: ${EMAIL_ACCOUNT:-fcristallocalagnoli@uma.es}
    env_file:
      - path: ./aws-secrets.env
        required: false
    ports:
      - "8000:80"
```

```

    expose:
      - 8000
    depends_on:
      - database
frontend:
  build:
    context: ./frontend
    target: develop
    args:
      - BUILD_MODE=docker
  ports:
    - "80:4200"
  expose:
    - 80
  depends_on:
    - backend
volumes:
  db-data:

```

Source Code 7: Docker compose file

B.3.1. Servicio *database*

El servicio *database* define el comportamiento del contenedor que albergará la base de datos MongoDB en la que se basa la aplicación. Tal contenedor se creará a partir de la imagen oficial de MongoDB, *mongo*, concretamente la versión *6.0.13-jammy*.

El resto de opciones significan lo siguiente:

- **restart:** la opción ‘always’ indica al contenedor que se reinicie siempre que ocurra algún fallo interno.
- **ports:** mapea el puerto 27017 del host al puerto 27017 del contenedor.
- **expose:** sirve para documentar qué puertos están expuestos, el 27017 en este caso.
- **volumes:** define dos volúmenes de datos
 - El mapeo `db-data:/data/db` asocia un volumen persistente para almacenar los datos de la aplicación.

- La segunda definición mapea el fichero `mongo-init.js` (presente en la carpeta del backend) a la carpeta de `/docker-entry-point-initdb/`, donde se almacenará con acceso únicamente de lectura. Este fichero es de gran importancia, pues es el encargado de configurar la base de datos la primera vez que arranca el contenedor.

Pasamos ahora a las variables de entorno:

- **DB_DATABASE:** define el nombre de la base de datos por defecto.
- **DB_ADMIN_USER:** define el nombre de usuario administrador por defecto.
- **DB_ADMIN_PASSWORD:** define la contraseña de administrador por defecto.
- **DB_DUMMY_ACCOUNT:** crea un usuario con el email proporcionado como *dummy account*.

B.3.2. Servicio *backend*

El servicio *backend* define el contenedor que correrá la API REST que alimenta la plataforma web. Maneja diferentes *endpoints* para tratar con cuentas de usuario, proveedores y sistemas. Su imagen se construye a partir del Dockerfile ubicado en la carpeta `backend/` del proyecto (código en 5).

Detallando sus opciones tenemos que:

- **env_file:** especifica un fichero que contiene variables de entorno. En este caso, el archivo `aws-secrets.env`.
 - Las variables que se están inyectando son unas claves de AWS usadas para inicializar la información de este tercero. Si no se incluye el fichero `aws-secrets.env`³¹, se omitirá de la ejecución.
- **ports:** mapea el puerto 8000 del host al puerto 80 del contenedor.
- **expose:** sirve para documentar qué puertos están expuestos, el 8000 en este caso.

³¹Se proporciona un ejemplo en el repositorio GitHub: <https://github.com/fcristallocagnoli/Quantum-Proxy/blob/master/aws-secrets.env.sample>. Remover sufijo 'sample' y rellenar con valores.

- **depends_on**: permite controlar el orden en que se crearán los contenedores. Al indicar una dependencia, un servicio no podrá crearse hasta que aquellos de los que dependa se hayan iniciado correctamente. Este servicio depende de *database*.

Pasamos ahora a las variables de entorno:

- **FRONTEND_URL**: define la dirección donde se encontrará el servicio de *frontend*. El puerto de esta dirección deberá ser el que exponga el frontend, en este caso como es el 80 (predeterminado de HTTP), lo podemos omitir.
- **DB_URI**: define la URI que servirá de conexión a la base de datos. Los atributos entre ‘:’ (admin:admin) deben corresponder a las claves **DB_ADMIN_USER** y **DB_ADMIN_PASSWORD** declaradas en el servicio de base de datos.
- **DB_PORT**: define el puerto de la conexión con la base de datos. Este debe corresponderse con el puerto expuesto en el servicio de la base de datos.
- **DUMMY_ACCOUNT**: define la cuenta de correo que se usará como *dummy account*³².
- **EMAIL_SENDER** y **EMAIL_APP_PASSWORD**: son variables usadas para el servicio de emails automatizados³³. **EMAIL_SENDER** define la cuenta en nombre de quien se enviarán los correos; **EMAIL_APP_PASSWORD** indica la contraseña de aplicación, necesaria para enviar emails de manera automática.
 - Para una mayor facilidad y simplicidad en el despliegue local, se proporcionan tanto la cuenta de correo como la contraseña de aplicación presentes en el fichero `docker-compose`. En un entorno real esto constituiría una pérdida de privacidad y confidencialidad, no se deben exponer ningún tipo de credenciales en el código.
- **EMAIL_ACCOUNT**: define la cuenta de correo a la que le serán enviados los correos con informes de posibles errores, dado el caso que falle algo en la aplicación.

³²Cuenta automatizada para obtener datos de proveedores y sistemas.

³³Ya sea para enviar emails de verificación de cuenta, o para informar de errores en la aplicación

B.3.3. Servicio *frontend*

Por último, el servicio *frontend* define el contenedor que servirá la aplicación web desarrollada en Angular. La imagen asociada se construye a partir del Dockerfile presente en la carpeta *frontend/* del proyecto, el cual vemos a continuación en los fragmentos de código 8, 9 y 10. Donde a través de la configuración de *compose*, escogemos construir la etapa *develop* y con el argumento *BUILD_MODE* puesto a 'docker'.

Detallando sus opciones tenemos que:

- **ports:** mapea el puerto 80 del host al puerto 4200 del contenedor.
- **expose:** sirve para documentar qué puertos están expuestos, el 80 en este caso.
- **depends_on:** permite controlar el orden en que se crearán los contenedores. Al indicar una dependencia, un servicio no podrá crearse hasta que aquellos de los que dependa se hayan iniciado correctamente. Este servicio depende de *backend*.

Como el Dockerfile es bastante grande y algo complejo, lo fragmentaremos en sus diferentes etapas para poder comprenderlo de manera sencilla.

- **Build stage:** parte de una imagen de *Node* y se encarga de instalar las dependencias del proyecto Angular, así como de construir el proyecto en base al valor del argumento *BUILD_MODE*³⁴. Esta etapa se ejecuta siempre.

```
# Selecciona Node como imagen base, versión lts-alpine
FROM node:lts-alpine AS build

# Establece el directorio de trabajo en /app
WORKDIR /app

# develop como build predeterminada
ARG BUILD_MODE=develop

# Copia los archivos package.json y package-lock.json
COPY package*.json .

# Instala las dependencias
RUN npm ci
```

³⁴Este argumento sirve para indicar internamente a la aplicación la ruta del backend.

```
# Copia el código del frontend
COPY . .
# Ejecuta el comando especificado en el argumento BUILD_MODE
RUN npm run ${BUILD_MODE}
```

Source Code 8: Etapa 'build', Dockerfile para el frontend

- **Develop stage:** parte de la etapa anterior *build*. Se encarga de instalar la CLI de Angular y servir la aplicación web. Esta etapa está pensada para el desarrollo.

```
# Selecciona la etapa build como base
FROM build as develop

# Instala Angular CLI
RUN npm install -g @angular/cli@17

# Expone el puerto 4200
EXPOSE 4200

# Indica el comando que se ejecutará al iniciar el contenedor
CMD [ "ng", "serve", "--host", "0.0.0.0" ]
```

Source Code 9: Etapa 'develop', Dockerfile para el frontend

- **Product stage:** parte de una imagen de *Nginx*, y se encarga copiar los archivos generados en la etapa *build* para servirlos como sitio web estático. Esta etapa está pensada para un despliegue optimizado usando *Nginx*.

```
# Selecciona Nginx como imagen base
FROM nginx:latest as product

# Copia el archivo de configuración de Nginx
COPY nginx.conf /etc/nginx/conf.d

# Copia los archivos generados en la etapa build a la carpeta de
  archivos estáticos de Nginx
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html

# Expone el puerto 80
```

EXPOSE 80

```
# Indica el comando que se ejecutará al iniciar el contenedor  
CMD [ "nginx", "-g", "daemon off;" ]
```

Source Code 10: Etapa 'product', Dockerfile para el frontend

B.4. Manejar los servicios con Docker Compose

Una vez se ha comprendido la configuración del fichero docker-compose y los correspondientes Dockerfiles, retomamos el proceso de instalación.

En este punto deberíamos encontrarnos dentro de la carpeta `Quantum-Proxy/`. Aquí, abriremos una terminal, y desplegaremos el entorno mediante comandos.

B.4.1. Construcción de los servicios

Trabajar con Docker Compose ofrece grandes ventajas a la hora del desarrollo, tal es así que únicamente precisaremos de un comando para arrancar y poner en marcha la aplicación.

```
$ docker compose up -d
```

Source Code 11: Comando para iniciar la aplicación

Una vez comience a ejecutarse el comando 11, lo primero que hará será construir las imágenes correspondientes al backend y frontend, y descargar de DockerHub la imagen usada para la base de datos. Esto es debido a que la primera vez que se arranque la aplicación, Docker no tiene registro de ninguna de las imágenes de los contenedores, por lo que deberá generarlas antes de crear los contenedores. Puede demorar un poco, pero una vez se hayan obtenido las imágenes, no tendrá que volver a hacerlo.

Tan pronto como se creen las imágenes, Docker comenzará a levantar los contenedores uno a uno, siguiendo el orden de dependencias visto en 7 hasta que estén todos activos.

B.4.2. Acceso a la plataforma recién desplegada

Ya con todos los servicios corriendo, podremos acceder a los servicios mediante los siguientes enlaces:

- En <http://localhost> encontraremos la plataforma web.
- En <http://localhost:8000> encontraremos la API REST.
- En <http://localhost:27017> encontraremos la conexión a la base de datos MongoDB, a la cual podremos acceder con *mongosh* o *MongoDB Compass*, pues no dispone de interfaz web.

B.4.3. Terminación de los servicios

Si se desea acabar con las instancias creadas, basta con ejecutar el comando 12 a continuación. Dicho comando detendrá los contenedores “amablemente” y luego los eliminará.

```
$ docker compose down
```

Source Code 12: Comando para terminar la aplicación

Opcionalmente se puede añadir la opción `-v`, que también borrará el volumen creado para almacenar los datos. Si no se especifica, cada vez que se levante la aplicación los datos almacenados en el servicio *database* persistirán.

Y como ya se expuso anteriormente, las imágenes ya han sido generadas, lo único que se elimina son los contenedores. De este modo, si se vuelve a levantar la aplicación con el comando 11 se ejecutará mucho más rápido.

B.4.4. Siguientes pasos a tomar

Desplegar los servicios es solo el principio, una vez tenemos toda la aplicación corriendo en local, hay una serie de consideraciones a tener en cuenta para terminar su configuración:

1. La plataforma se inicializa con dos cuentas predeterminadas:

- Email: `admin@test.com`, Contraseña: `admintesting`
- Email: `<DB_DUMMY_ACCOUNT>`³⁵, Contraseña: `dummy-password`

La primera cuenta posee privilegios de administrador para cualquier tarea oportuna de administración. Mientras que la segunda supone la denominada *dummy account*. Esta

³⁵El contenido de dicha variable de entorno. Por defecto será `quantum.proxy@gmail.com`, pero se puede modificar por el usuario en el fichero `docker-compose`.

cuenta *dummy* deberá ser actualizada con las API keys correspondientes para poder obtener los datos de los proveedores y sus máquinas correctamente.

The image shows a dark-themed user interface for managing API keys. It features three sections, each with a 'Platform' label, a 'Key' label, and a 'Value' label. The first section is for 'ionq' with a 'TOKEN' key. The second section is for 'ibm' with a 'TOKEN' key. The third section is for 'amazon_braket' with three keys: 'AWS_ACCESS_KEY_ID', 'AWS_SECRET_ACCESS_KEY', and 'AWS_REGION'. Each key has a corresponding input field with a placeholder '<YOUR_VALUE>'. To the right of each input field are two buttons: a red button with a minus sign and a grey button with a plus sign. At the bottom of the third section, there are also two such buttons.

Platform	Key	Value		
ionq	TOKEN	<YOUR_VALUE>	-	+
ibm	TOKEN	<YOUR_VALUE>	-	+
amazon_braket	AWS_ACCESS_KEY_ID	<YOUR_VALUE>		
	AWS_SECRET_ACCESS_KEY	<YOUR_VALUE>		
	AWS_REGION	<YOUR_VALUE>	-	+

Figura 51: Ejemplo de API keys requeridas para obtener los datos

2. Una vez rellenas las API keys, pueden inicializarse los sistemas desde el panel de administrador dentro de la plataforma.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga